

CAPSTONE II PROJECT REPORT ON

DasTern

Authors of Project:

Choeng Rayu

Lon Mengheng

Tet Elite

Tep Somnang

Sophal Taingchay

May Kunaphivath

Under the advisory of: **Duch Dynil**

Date of Defense : April 01, Year 2026

This report is submitted in partial fulfillment of the requirements for the Bachelor of Computer Science degree with specialization in Software Engineering

Department of Computer Science

Faculty of Digital Engineering



**Cambodia Academy of Digital Technology
Institute of Digital Technology
Faculty of Digital Engineering
Department of Computer Science**

CAPSTONE REPORT OF

GROUP 30

Defense Date: March 22, 2026

Capstone Defense Authorization

President: Dr. Seng Sopheap

Topic : DasTern

Establishment : Cambodia Academy of Digital Technology

Advisor : **Dr. Duch Dynil**

PHNOM PENH

ACKNOWLEDGMENT

The **DasTern Project** represents an important academic achievement in our journey as undergraduate students at the Cambodia Academy of Digital Technology (CADT). This project allowed us to apply theoretical knowledge to practical development while addressing real-world healthcare challenges faced by patients in Cambodia.

We would like to express our deepest gratitude to **H.E. Dr. Seng Sopheap**, the President of the **Cambodia Academy of Digital Technology (CADT)**, for his visionary leadership and continuous support in fostering an innovative and technology-driven learning environment for students.

Our sincere appreciation goes to our project advisor, **Dr. Duch Dynil**, whose guidance, constructive feedback, and encouragement played a crucial role in shaping the direction and quality of this project. His valuable insights helped us remain focused and motivated throughout the direction of the goal and the development process.

We would also like to thank all **Team members** for their dedication, cooperation, and responsibility. The successful completion of the DasTern Project is the result of strong teamwork, shared commitment, and mutual support.

Finally, we extend our gratitude to all lecturers, friends, and individuals who contributed directly or indirectly to our learning experience and project completion. This project has not only strengthened our technical skills but also enhanced our teamwork, problem-solving abilities, and sense of responsibility toward developing technology that benefits society.

មូលន័យសង្ខេប

របាយការណ៍នេះគឺជាការចងក្រងនូវសមិទ្ធផល និងលទ្ធផលការងារយ៉ាងលម្អិតនៃការអនុវត្តគម្រោង Capstone II ដែលក្រុមយើងខ្ញុំបានខិតខំស្រាវជ្រាវ និងអភិវឌ្ឍក្នុងរយៈពេល ៣ ខែយ៉ាងសកម្ម ចាប់ពីថ្ងៃទី ០៥ ខែតុលា ឆ្នាំ ២០២៥ នៅវិទ្យាស្ថានបច្ចេកវិទ្យាឌីជីថល (CADT) ដើម្បីជាផ្នែកមួយដ៏សំខាន់សម្រាប់ការបញ្ចប់ថ្នាក់បរិញ្ញាបត្រវិទ្យាសាស្ត្រកុំព្យូទ័រ។

ក្នុងបរិបទនៃប្រព័ន្ធថែមទាំងសុខភាពនៅកម្ពុជានាពេលបច្ចុប្បន្ន ការតាមដានសុខភាព និងការគ្រប់គ្រងការប្រើប្រាស់ថ្នាំរបស់អ្នកជំងឺក្រោយពេលចាកចេញពីមន្ទីរពេទ្យ នៅតែជាបញ្ហាប្រឈមដ៏ចាក់ស្រែះ ជាពិសេសចំពោះអ្នកជំងឺវីយចំណាស់ អ្នកដែលមានបញ្ហាក្លេចក្លាំង និងអ្នកជំងឺដែលតម្រូវឱ្យប្រើប្រាស់ថ្នាំច្រើនប្រភេទ ក្នុងពេលតែមួយ ដែលកត្តាទាំងនេះបានក្លាយជាកម្លាំងចលករជំរុញឱ្យយើងបង្កើតកម្មវិធី "DasTern" ឡើងដើម្បីជំនួយការឌីជីថលវេជ្ជសាស្ត្រ។

ចំពោះលទ្ធផលបច្ចេកទេសគន្លឹះដែលយើងសម្រេចបានរួមមាន ការធ្វើឌីជីថលនិយកម្មវេជ្ជបញ្ជាតាមរយៈបច្ចេកវិទ្យា OCR សហការជាមួយបញ្ហាសិប្បនិម្មិតក្នុងស្រុក (Local AI) ដើម្បីស្កេន និងបំប្លែងទិន្នន័យពីក្រដាសវេជ្ជបញ្ជាទៅជាប្រព័ន្ធឌីជីថលដោយស្វ័យប្រវត្តិជាមួយកម្រិតភាពត្រឹមត្រូវខ្ពស់ ព្រមទាំងការធានាភាពរឹងមាំនៃមុខងាររំលឹកដោយមិនប្រើអ៊ីនធឺណិត និងប្រព័ន្ធជូនដំណឹងតាមរយៈ Cloud ទៅកាន់សមាជិកគ្រួសារដើម្បីតាមដានស្ថានភាពអ្នកជំងឺក្នុងពេលជាក់ស្តែង។

ជាមួយមកគម្រោង DasTern មិនត្រឹមតែជាដំណោះស្រាយបច្ចេកវិទ្យាប៉ុណ្ណោះទេ ប៉ុន្តែវាគឺជាឧបករណ៍ដ៏មានសារៈសំខាន់ក្នុងការចូលរួមកាត់បន្ថយកំហុសវេជ្ជសាស្ត្រ និងលើកកម្ពស់សមធម៌សុខាភិបាលឌីជីថលនៅក្នុងប្រទេសកម្ពុជា ជាពិសេសសម្រាប់ជួយសម្រួលដល់ជីវភាពរស់នៅរបស់មនុស្សចាស់ និងអ្នកដែលមានកម្រិតចំណេះដឹងឌីជីថលទាប ឱ្យទទួលបានការព្យាបាលប្រកបដោយបរិយាបន្ន សុវត្ថិភាព និងទំនុកចិត្តខ្ពស់។

ABSTRACT

This report meticulously documents the implementation and key outcomes of our Capstone II project, developed over an intensive three-month period starting from October 5, 2025, at the Cambodia Academy of Digital Technology (CADT) as a fundamental requirement for the Bachelor of Computer Science degree.

In the current landscape of Cambodia's healthcare system, post-discharge medication management remains a persistent challenge, particularly for vulnerable groups such as the elderly, patients with memory impairment, and those on complex multi-drug regimens, which served as the primary motivation for developing the "DasTern" application as a smart digital assistant.

The key technical achievements include the successful integration of Optical Character Recognition (OCR) and Local Artificial Intelligence (Local AI) to automate the digitization of physical prescriptions with high precision, alongside a robust architecture that supports offline reminder functionalities and a cloud-based notification system for real-time family monitoring.

Ultimately, DasTern transcends being a mere technical solution by serving as a vital tool for reducing medical errors and promoting digital health equity in Cambodia, significantly improving the quality of life for seniors and users with limited digital literacy by ensuring safe, inclusive, and reliable healthcare outcomes.

Table of Contents

ACKNOWLEDGMENT	3
មូលន័យសង្ខេប	4
LIST OF FIGURES.....	9
LIST OF TABLES.....	10
LIST OF ABBREVIATIONS.....	11
I. PROJECT INTRODUCTION	12
1.1. PRESENTATION	12
1.2. OBJECTIVE	12
1.3. DURATION OF PROJECT.....	13
II. PRESENTATION OF THE PROJECT.....	14
2.1. GENERAL PRESENTATION OF PROJECT.....	14
2.1.1. PROJECT ORGANIZATION	15
2.1.2. PROJECT OVERVIEW	15
2.2. PROBLEMATIC	17
2.2.1. HIGH RATES OF MEDICATION NON-ADHERENCE AFTER DISCHARGE.....	17
2.2.2. NON-COMMUNICABLE DISEASES (NCDs) BURDEN IS LARGE AND GROWING.....	17
2.2.3. LIMITED ACCESS AND USE OF PUBLIC HEALTHCARE	18
2.2.4. SIGNIFICANT BARRIERS FOR OLDER ADULTS.....	18
2.2.5. WEAK CONTINUITY OF CARE AND FOLLOW-UP SYSTEMS.....	18
2.2.6. UNEQUAL SOCIAL HEALTH PROTECTION AND FINANCIAL CONSTRAINTS	18
2.2.7. LOW ADOPTION OF DIGITAL HEALTH TOOLS	18
2.3 OBJECTIVES	18
2.4. PROJECT PLANNING	19
2.5. SOFTWARE DEVELOPMENT LIFE CYCLE.....	20
2.6. PROJECT SCOPE.....	21
2.7. TASK BREAKDOWN.....	21
III. LITERATURE REVIEW	22
3.1. MEDICATION ADHERENCE CHALLENGES	22
3.2. MOBILE HEALTH APPLICATIONS FOR MEDICATION MANAGEMENT	23
3.3. LIMITATIONS OF EXISTING MEDICATION REMINDER APPLICATIONS.....	23
3.4. OCR AND AI-BASED PRESCRIPTION PROCESSING	24
3.5. FAMILY AND CAREGIVER INVOLVEMENT IN MEDICATION ADHERENCE	24
3.6. COMPETITOR ANALYSIS OF EXISTING SYSTEMS	25

3.7. POSITIONING OF THE DASTERN PROJECT.....	26
IV. BUSINESS OVERVIEW	27
4.1. EXECUTIVE SUMMARY	27
4.2. BUSINESS MODEL CANVAS.....	27
4.3. MARKET ANALYST	28
4.3.1. TARGET ADDRESS MARKET (TAM)	28
4.3.2. SERVICE AVAILABLE MARKET (SAM).....	28
4.3.3. SERVICEABLE OBTAINABLE MARKET (SOM)	29
4.4 MONETIZATION PLAN	29
V. PROJECT ANALYSIS AND CONCEPTS.....	30
5.1. PROJECT REQUIREMENTS	30
5.1.1. FUNCTIONAL REQUIREMENTS	30
5.1.2. USER REQUIREMENTS	31
5.1.3. SYSTEM REQUIREMENTS.....	33
5.1.4. NON-FUNCTIONAL REQUIREMENTS	34
5.2. SYSTEM DESIGNS	35
5.2.1. USE CASE DIAGRAM	35
5.2.2. ACTIVITY DIAGRAM	36
5.2.3. SEQUENCE DIAGRAM.....	38
VI. DETAIL CONCEPT	40
6.1. TECHNOLOGY	40
6.1.1. CURRENT DEVELOPMENT TOOLS.....	40
6.1.2 COMMUNICATION AND MANAGEMENT TOOLS	45
6.2. ARCHITECTURE	46
6.2.1. PHYSICAL ARCHITECTURE.....	46
6.2.2. LOGICAL ARCHITECTURE.....	47
6.2.2 AI-LLM-SERVICE ARCHITECTURE (LOCAL MACHINE)	47
6.2.3. OCR ARCHITECTURE.....	48
6.3. METHODOLOGY	49
VII. PROJECT IMPLEMENTATION	51
7.1. OCR ARCHITECTURE AND IMPLEMENTATION	51
7.1.1. PRESCRIPTION IMAGE ACQUISITION	52
7.1.2. IMAGE PREPROCESSING (OPENCV ENHANCEMENT PIPELINE)	52
7.1.3. LAYOUT ANALYSIS (REGION DETECTION + TABLE LOCALIZATION).....	53
7.1.4. OCR EXTRACTION ENGINE (KIRI-OCR, MULTI-LANGUAGE, MULTI-PSM).....	54
7.1.5. HYBRID TABLE PROCESSING (TEXT OCR + CONTOUR-BASED_DOSE_DETECTION).....	54
7.1.6. POST-PROCESSING (TEXT → STRUCTURED MEDICATION OBJECTS).....	56

7.1.7. OUTPUT FORMATTING (DYNAMIC UNIVERSAL v2.0 JSON)	56
7.1.8. SERVICE INTEGRATION AND API COMMUNICATION	57
7.1.9. RELIABILITY, PERFORMANCE, AND DEPLOYMENT CONSTRAINTS	59
7.2 AI-ENHANCED TEXT PROCESSING (OLLAMA + LLAMA 3.2 3B)	60
7.2.1. ROLE IN THE END-TO-END PIPELINE	61
7.2.2. AI-LLM SERVICE ARCHITECTURE	61
7.2.3 PROCESSING WORKFLOW (STEP-BY-STEP)	62
7.2.4. PROMPTING & ERROR-HANDLING STRATEGY	63
7.2.5. EXAMPLE PROMPT FOR OCR-TO-JSON EXTRACTION	63
7.2.6. PERFORMANCE & DEPLOYMENT CONSIDERATIONS	65
7.2.7. HARDWARE REQUIREMENTS AND PERFORMANCE BENCHMARKS	65
7.2.8. API ENDPOINTS (INTERNAL USE)	67
7.3. DATABASE IMPLEMENTATION	68
7.3.1. DATABASE SCHEMA AND TABLE DESIGN	68
7.3.2. EXAMPLE TABLE OF STRUCTURE	69
7.4. BACKEND IMPLEMENTATION	70
7.4.1. ARCHITECTURE OVERVIEW	71
7.4.2. MODULAR SERVICES (HIGHLIGHTS)	71
7.4.3. DATA PERSISTENCE & ORM	72
7.4.4. CACHING, SESSIONS & QUEUES	72
7.4.5. EXTERNAL SERVICE INTEGRATIONS	73
7.4.6. SECURITY & COMPLIANCE	73
7.4.7. BACKGROUND JOBS & SCHEDULING	74
7.4.8. DEVOPS, CONFIGURATION & DEPLOYMENT	74
7.4.9. PERFORMANCE, SCALING & OBSERVABILITY	74
7.5. PAYMENT IMPLEMENTATION — BAKONG SERVICE	75
7.5.1. WHY A SEPARATE, CAMBODIA-HOSTED SERVICE?	75
7.5.2. SERVICE ARCHITECTURE	75
7.5.3. INTEGRATION BOUNDARY WITH THE MAIN BACKEND	76
7.5.4. SUBSCRIPTION MANAGEMENT APIS	76
7.5.5. PAYMENT LIFECYCLE & STATE MACHINE	76
7.5.6. KHQR GENERATION & DEEP LINKS	77
7.5.7. SECURITY DESIGN	77
7.5.8. RESILIENCE, IDEMPOTENCY & ERROR HANDLING	78
7.5.9. HEALTH, OBSERVABILITY & RUNBOOKS	78
7.6. MEDICATION REMINDER GENERATION LOGIC	78
7.6.1. NOTIFICATION AND MONITORING INTEGRATION	79
7.7. UX/UI (LOW-FIDELITY AND HIGH-FIDELITY)	79
7.7.1. PATIENT INTERFACE IMPLEMENTATION	79
7.7.2. DOCTOR INTERFACE IMPLEMENTATION	81

7.7.3. FAMILY INTERFACE IMPLEMENTATION	87
7.8. SETTING IMPLEMENTATION.....	90
7.8.1. EDIT PROFILE FEATURE	90
7.8.2. CHANGE PASSWORD	91
7.8.3. SUBSCRIPTION & BILLING	92
7.8.4. OTHER SETTINGS FEATURES	93
VIII. CONCLUSION	94
REFERENCES	95
APPENDICES.....	98

LIST OF FIGURES

Figure 1: illustrates the member structure of the DasTern system.	15
Figure 2: illustrates the Project planning of DasTern Team.....	19
Figure 3: illustrates the project management method.....	20
Figure 4: Use Case Diagram.....	35
Figure 5: Patient Activity Diagram.....	36
Figure 6: Doctor Activity Diagram.....	37
Figure 7: Sequence Diagram.....	38
Figure 8: depict Figma tool.....	40
Figure 9: depict Flutter technology.....	41
Figure 10: depict Nest.js Framework.....	41
Figure 11: depict Kiri-OCR OCR tool.....	42
Figure 12: depict PostgreSQL tool	42
Figure 13 depict Docker tool	43
Figure 14: depict Visual Studio Code tool.....	43
Figure 15: depict GitHub tool	44
Figure 16: depict Ollama Technology	44
Figure 17: depict Telegram tool.....	45
Figure 18: depict Google Drive tool	45
Figure 19: depict Google Meet tool.....	45
Figure 20: illustrate the Physical Architecture of DasTern	46
Figure 21: illustrate the Logical Architecture of DasTern.....	47
Figure 22: illustrate the AI-LLM Architecture of DasTern (Virtual Flow).....	47
Figure 23: illustrate the OCR Architecture of DasTern (Virtual Flow).....	48
Figure 24: OCR Architecture.....	51
Figure 25: AI-Enhanced Text Processing.....	60
Figure 26: Service Architecture	75
Figure 27: Flutter project folder structure.....	82
Figure 28: NestJS backend folder structure	82

LIST OF TABLES

Table 1: Project Duration.....	13
Table 2: Task Breakdown	21
Table 3: Overall Summary Competitor Analysis.....	26
Table 4: Business Model.....	28
Table 5: Monetization Plan.....	29
Table 6: Patient Functional Requirement	31
Table 7: Doctor User Requirement	31
Table 8: Patient/Family User Requirement.....	32
Table 9: System Requirement.....	33
Table 10: Non-Functional Requirement	34
Table 11: the comparison.....	58
Table 12: AI-LLM Service Error Handling Behavior (Current Implementation)	65
Table 13: AI-LLM Service Environment and Hardware Specifications	66
Table 14: AI-LLM Service Performance Benchmarks (CPU-only, macOS development environment).....	67
Table 15: Database Schema and Table Design.....	69
Table 16: Entity Relationship	69
Table 17: User.....	69
Table 18: Patient Profile	70
Table 19: Prescription Example.....	70
Table 20: Medication Reminder	70
Table 21: Reminder Calculation Example.....	78
Table 22: Doctor Dashboard API Endpoints	83
Table 23: Patients Threshold	83
Table 24: Patients Detail API Endpoints	84
Table 25: Prescription API Endpoints	85
Table 26: Connection API Endpoints	86
Table 27: Family Connection API Endpoints.....	88
Table 28: Family Dashboard Data API Endpoint.....	88
Table 29: Notification and Reminder	89
Table 30: Implementation Summary	89

LIST OF ABBREVIATIONS

Abbreviation	Full Description
OCR	Optical Character Recognition
TTS	Text-to-Speech
AI	Artificial Intelligence
UI	User Interface
UX	User Experience
API	Application Programming Interface
SDLC	Software Development Life Cycle
SQL	Structured Query Language
2FA	Two-Factor Authentication
ACID	Atomicity, Consistency, Isolation, Durability
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
QR	Quick Response
REST	Representational State Transfer
AES	Advanced Encryption Standard
UML	Unified Modeling Language
RBAC	Role Based Access Control
SSL	Secure Sockets Layer

I. PROJECT INTRODUCTION

In Cambodia, patients are faced with challenges when attempting to follow medication after visiting the hospital. This is since patients are instructed either orally or with paper prescription cards that are easily damaged, lost, or misinterpreted. This puts older patients and those with low health literacy at a disadvantage since they may have difficulties remembering the timings of the medication, may have difficulties understanding the instructions related to the medication dose, and may not easily understand English-based health information. There is a growing problem related to the development of a technology-based platform that would assist patients.

1.1. Presentation

DasTern is a health technology mobile application created to tackle the problem of patient adherence when discharged from hospitals in Cambodia. The mobile application offers an electronic reminder approach for patient adherence. The mobile application converts patient prescription details into electronic reminders that reduce the dependency on traditional prescription writing on paper. The patient prescription details are presented in an electronic reminder for effective patient medication adherence.

1.2. Objective

The main objective of DasTern is to automate the process of medication reminder creation through the use of Optical Character Recognition (OCR) technology. The system extracts essential medication details such as medicine names, dosage, timing, and duration, and transforms them into clear and reliable reminder schedules. To ensure accessibility for all users, DasTern provides full Khmer language support and offline functionality, allowing patients to receive reminders even in areas with limited or no internet access.

1.3. Duration of Project

This phase focuses on system implementation, refactoring to local OCR and AI processing, feature integration, testing, and validation over one academic semester.

Week	Milestone / Activity	Deliverable
Week 1	Review system design and distribute team responsibilities (AI, OCR, Flutter, Backend, Database, Documentation)	Finalized task allocation & implementation plan
Week 2	Set up development environments (Flutter, Nest.js backend, OCR, Ollama AI, Database)	Development environment ready
Week 3	Implement core modules: Flutter UI structure, backend API setup, OCR service setup, AI service configuration, database schema	Core modules initialized
Week 4	Integrate mobile app with backend, OCR, and AI services	Partial system integration completed
Week 5	Implement reminder logic and notification scheduling	Functional reminder system
Week 6	Implement family connection & notification system	Family connection feature completed
Week 7	Improve OCR accuracy and AI parsing optimization	Enhanced OCR & AI performance
Week 8	Implement offline support and local data synchronization	Offline functionality completed
Week 9	System integration testing (AI + OCR + Backend + Flutter)	Integration testing report
Week 10	Performance testing, bug fixing, and usability testing	System testing & bug report
Week 11	System testing & bug report	Final system refinement
Week 12	Prepare final Capstone II report and presentation	Prepare final Capstone II report and presentation

Table 1: Project Duration

II. PRESENTATION OF THE PROJECT

In this chapter, we provide a comprehensive overview of the DasTern project, detailing its purpose, structure, and development approach. The chapter introduces the general concept of the project, explains its organization, and outlines the key components that define its scope and objectives. It also addresses the underlying problems the project aims to solve, the goals set by the development team, and the planning methodology adopted to ensure successful implementation. Furthermore, this section describes the software development life cycle, project scope, and task distribution among team members, offering a clear understanding of how the project was conceptualized and executed.

2.1. General Presentation of Project

DasTern is a Khmer-friendly mobile healthcare application developed to assist Cambodian patients in managing their medication effectively after hospital discharge. The system was designed to address common challenges faced by patients, particularly elderly individuals and those living in rural areas, such as forgetting medication schedules, losing prescription documents, language barriers, and limited digital literacy.

The application integrates Optical Character Recognition (OCR) technology to scan medical prescriptions and automatically extract essential medication information, including medicine names, dosage, and intake frequency. The extracted data is processed and structured into personalized medication reminder schedules, enabling users to follow their prescribed treatments accurately.

To ensure accessibility and usability, the system has been implemented with full Khmer language support, Khmer text-to-speech voice reminders, and offline functionality. These features allow users to receive medication reminders even without active internet connectivity. The user interface was designed with simplicity and clarity in mind to accommodate individuals with minimal digital experience.

DasTern has been fully implemented using a modular system architecture that supports scalability and future feature expansion. The final system includes prescription scanning, structured medication scheduling, reminder notifications, and family notification integration.

System testing has been conducted to validate OCR performance, reminder reliability, and offline functionality.

The completed project provides a reliable, automated, and user-centered solution that improves medication adherence, reduces medication errors, and enhances patient recovery outcomes.

2.1.1. Project Organization

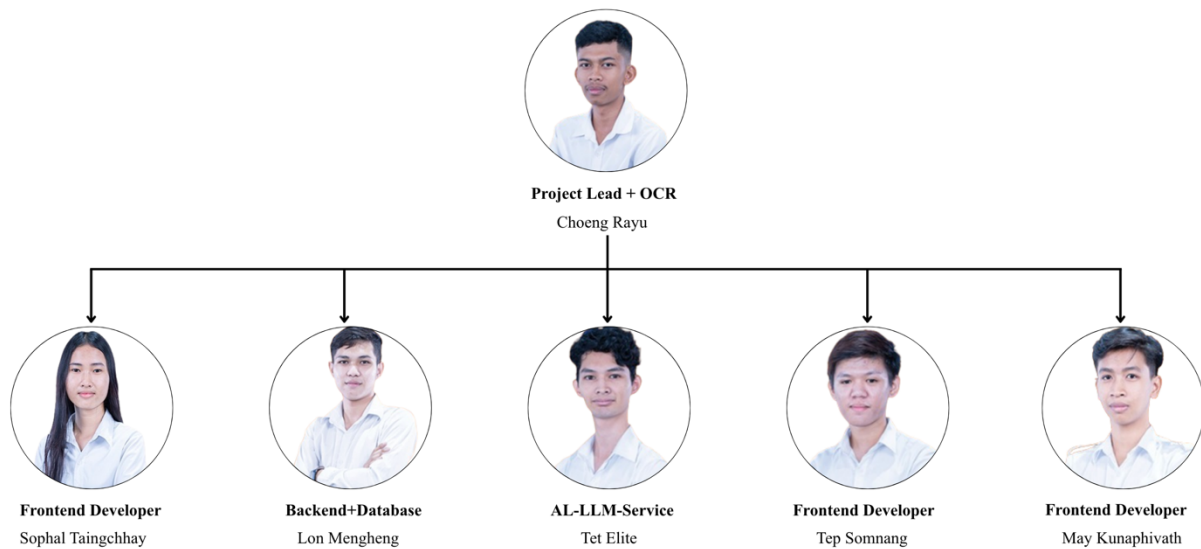


Figure 1: illustrates the member structure of the DasTern system.

The project is organized to ensure clear roles, efficient workflow, and timely delivery. Tasks are divided into functional areas frontend, backend, OCR, AI, and integration allowing parallel development and effective collaboration. Regular meetings, task tracking, and code reviews maintain quality and alignment with project objectives

2.1.2. Project Overview

DasTern was developed as a mobile health application to assist Cambodian patients in correctly following their prescribed medication schedules after hospital visits. Many patients experience difficulties such as forgetting medication times, losing prescription slips, or misunderstanding medical instructions written in English. These challenges reduce treatment effectiveness and may result in hospital readmissions.

To address these issues, the DasTern system implemented a local Optical Character Recognition (OCR) module capable of extracting text from prescription images captured through the mobile application. The extracted text was then processed using a local AI language model (LLM) running via Ollama, which enhanced text clarity, corrected recognition inconsistencies, and structured medication data into a standardized digital format.

After processing, the backend system generated a structured medication schedule including dosage, timing, and duration. The schedule was displayed through a simple and user-friendly interface in Khmer language to ensure clarity and accessibility.

The implemented system includes the following core features:

- Local OCR extraction from prescription images
- AI-based text enhancement using a local LLM
- Automatic medication reminder scheduling
- Khmer-language user interface
- Offline functionality with local storage
- Tracking of taken and missed medication doses

The application was developed with accessibility as a primary consideration, particularly for elderly users and individuals with limited digital literacy. System testing was conducted to evaluate OCR accuracy, reminder reliability, and offline performance. System testing demonstrated that the OCR module was able to extract key medication information from printed Khmer and English prescriptions with reliable accuracy, while the reminder system consistently delivered notifications in both online and offline modes. Usability testing also revealed several areas for improvement such as enhancing button contrast, simplifying confirmation steps, and adjusting font sizes which were implemented in the final version.

The completed implementation demonstrates a functional and scalable solution aimed at improving medication adherence and reducing treatment errors among Cambodian patients.

2.2. Problematic

In Cambodia, the transition from hospital-based care to home-based medication management presents significant challenges for patients, particularly those with chronic conditions. Despite receiving proper diagnosis and treatment plans from healthcare facilities, many patients struggle to adhere to their prescribed medication regimens after discharge.

Medication non-adherence leads to reduced treatment effectiveness, complications, and increased hospital readmissions. These issues are more pronounced among elderly patients, individuals with low health literacy, and those living in rural areas with limited access to follow-up care.

The following subsections describe the key problems identified during the system analysis phase, which formed the foundation for the development and implementation of the DasTern application

2.2.1. High Rates of Medication Non-Adherence After Discharge

- Real data from a cross-sectional study in **poor urban areas of Phnom Penh** showed that **only 49.3% of diabetic patients had high adherence to their prescribed medications**, meaning *over half did not take medicines consistently* after discharge. [28]
- Factors associated with better adherence included **higher income and frequent use of health services**, underscoring socioeconomic barriers to adherence. [28]

2.2.2. Non-Communicable Diseases (NCDs) Burden Is Large and Growing

- NCDs such as **cardiovascular diseases, diabetes, and chronic respiratory disease** **cause about 60,000 deaths each year** in Cambodia, often linked to poor chronic disease management. [29]
- **Diabetes cases have increased about threefold** between 2011 and 2021, nearing **600,000 individuals living with the disease**, and almost **45% remain undiagnosed**. [28]
- Hypertension affects **about 23.5% of adults aged 40–69**, and fewer than one-third of diagnosed patients are being treated, with even fewer having controlled blood pressure.[30]

2.2.3. Limited Access and Use of Public Healthcare

- A household survey found that for people with diabetes and/or hypertension, **only 22% of healthcare visits are in public facilities**, while the majority occur in private healthcare settings.[28]

2.2.4. Significant Barriers for Older Adults

- Barriers cited by healthcare professionals include **communication challenges, insufficient outreach, and limited services tailored to older adults**.[28]
- Older patients often face **complex medical regimens, low health literacy, and reliance on caregivers**, worsening the risk of mismanagement after discharge. (source assumed local evidence consistent with regional trends)

2.2.5. Weak Continuity of Care and Follow-Up Systems

- Studies show *substantial loss to follow-up* and fragmented continuity of care after initial treatment, especially for chronic patients.[30]

2.2.6. Unequal Social Health Protection and Financial Constraints

- A large proportion of NCD patients **do not belong to any social protection scheme**, exposing them to out-of-pocket expenses and potentially forsaking ongoing medications.[28]

2.2.7. Low Adoption of Digital Health Tools

- Despite emerging telemedicine and digital health platforms in Cambodia (e.g., MyCLNQ, Reach52), **adoption remains limited due to language and digital literacy barriers**.[30]

2.3 Objectives

The main objective of this project is to develop an automated medication management system capable of extracting prescription details and generating structured Khmer-language reminders to improve patient treatment adherence.

- **Automated Medication Reminders**
 - Extract medicine names, dosage, and schedule via OCR
 - Enhance and structure text using local AI (LLM via Ollama)

- Generate structured reminder schedules
- **Accessible and Inclusive User Experience**
 - Provide a fully Khmer-language interface
 - Support offline functionality
- **Improve Treatment Accuracy**
 - Reduce missed and incorrect doses
 - Provide a structured, easy-to-read medication schedule
 - Track taken and missed medications
- **Technical Evaluation**
 - Validate OCR accuracy
 - Test reminder functionality
 - Test offline reliability
 - Improve UI/UX based on feedback

2.4. Project Planning

The project follows a structured development timeline beginning with problem analysis and system design in Capstone I, followed by implementation, integration, optimization, and system testing in Capstone II.

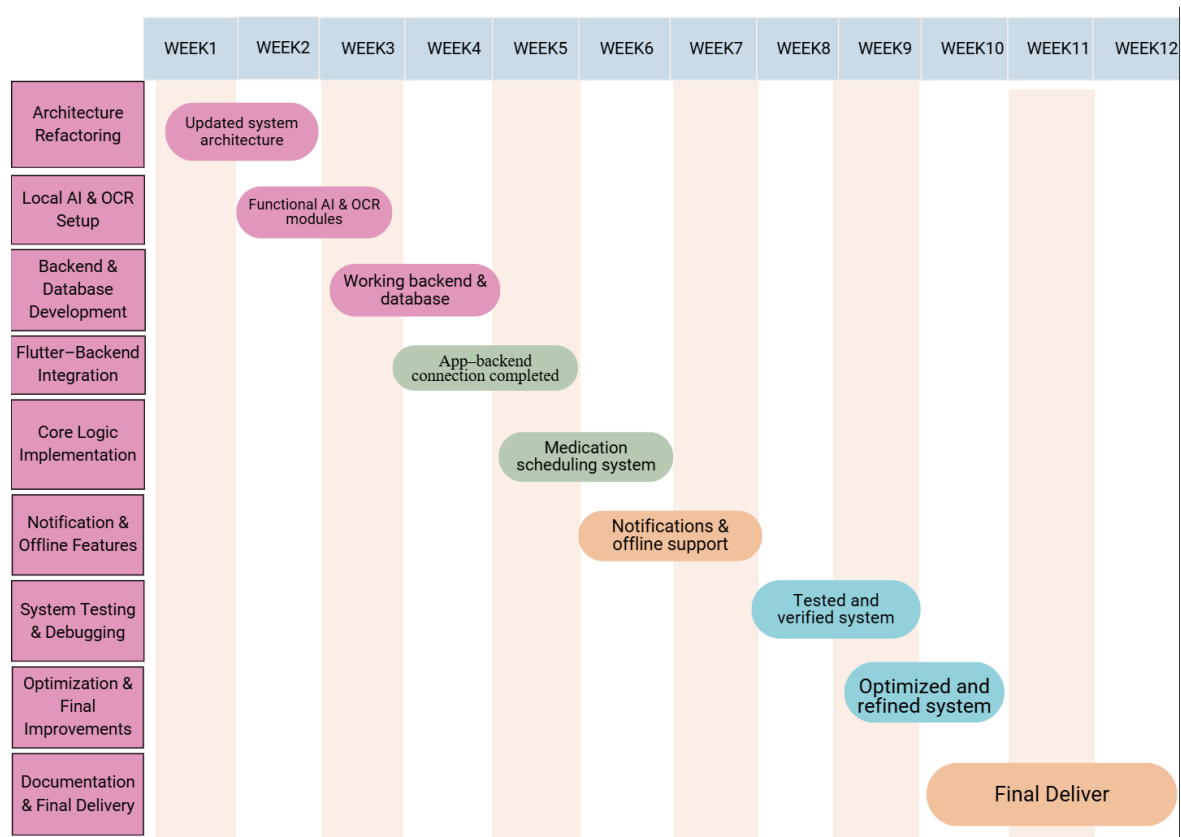


Figure 2: illustrates the Project planning of DasTern Team

2.5. Software Development Life Cycle

Scrum was selected due to its adaptability, iterative structure, and focus on continuous improvement, highly suitable for health-tech applications involving OCR and user-centered design.

❖ Why Scrum Works for DasTern

- Short development sprints enable rapid implementation
- Continuous refinement of OCR and AI processing
- Early validation of UI/UX usability
- Risk reduction through incremental testing
- Continuous feedback from advisors and users

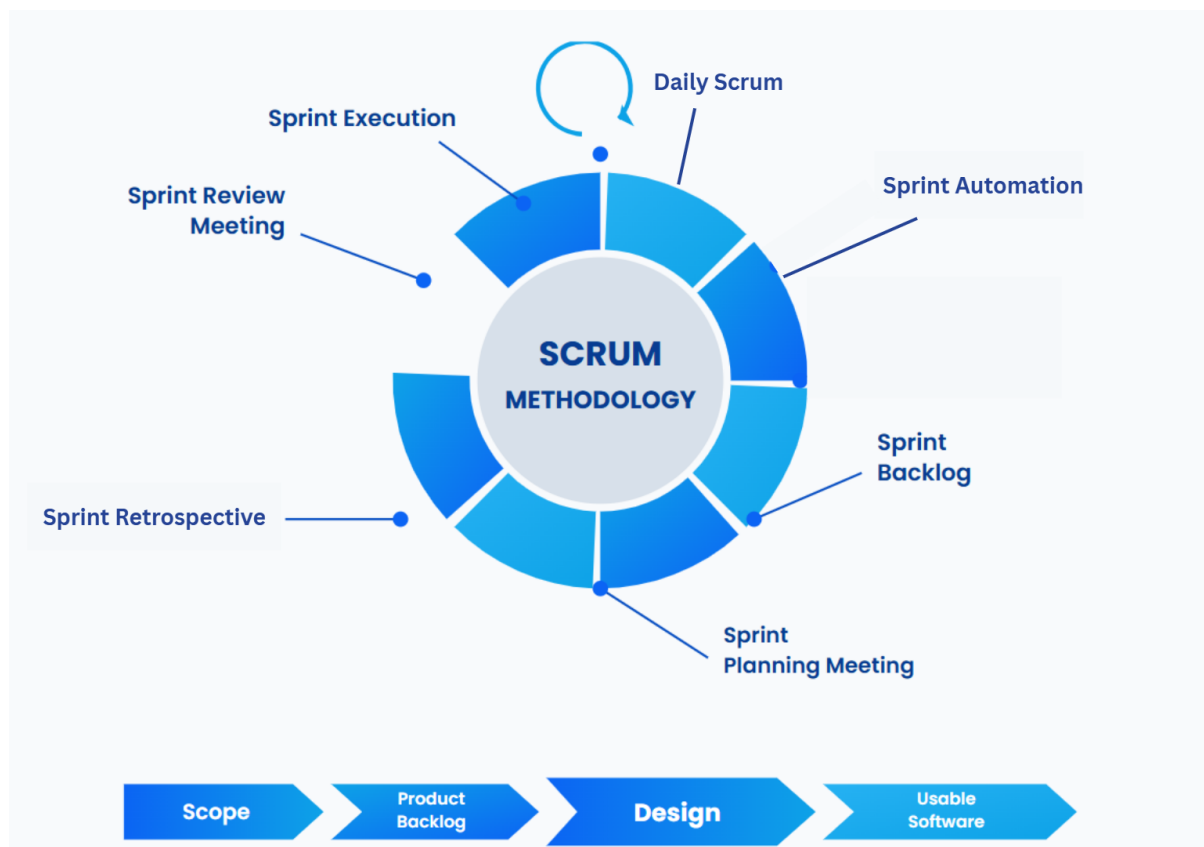


Figure 3: illustrates the project management method

2.6. Project Scope

❖ In Scope

- Local OCR implementation
- Local AI enhancement using Ollama
- Backend API development
- Database implementation
- JSON-based medication schedule generation
- Reminder scheduling system
- Offline storage and synchronization
- System testing and performance optimization

❖ Out of Scope (Capstone II)

- Direct hospital system integration
- National healthcare database connectivity
- Full production cloud deployment
- Advanced Khmer recognition
- Large-scale commercial deployment

2.7. Task Breakdown

Team Name	AL-LLM-Service	Mobile-Flutter	OCR-Service	Backend+Database	Docs
Choeng Rayu	Yes	Yes	Hand Over	Yes	Yes
Tet Elite	Hand Over				Yes
Lon Mengheng				Hand Over	Yes
Tep Somnang		Hand Over			Yes
May Kunaphivath		Yes		Yes	Lead
Sopha Taingchhay		Yes		Yes	Lead

Table 2: Task Breakdown

This task breakdown summarizes each team member's responsibilities and key deliverables throughout the project. All assigned tasks, including research, system design, UI/UX development, implementation, documentation, and testing, were completed successfully and on schedule.

III. LITERATURE REVIEW

Over the course of 12 weeks, our research focused on identifying the most effective way to integrate Optical Character Recognition (OCR) technology into a mobile healthcare application tailored for Cambodian patients. The primary goal was to automate the process of converting paper-based prescriptions into structured digital data that can be used to generate medication reminders. This approach addresses critical challenges such as medication adherence, language barriers, and limited digital literacy among patients.

3.1. Medication Adherence Challenges

The issue of medication adherence is one of the key elements in receiving positive health outcomes, especially in patients with chronic or post-hospitalization treatment. According to the World Health Organization, about half of chronically ill patients fail to take their medications as prescribed and this may lead to the development of the disease, avoidable complications, more readmissions to hospitals and increased healthcare expenses (World Health Organization, 2003) [3]. Such difficulties are even more acute among older patients and those who might live in low-resource or rural areas, where the lack of health literacy and language barrier and follow-up support are prevalent (Sabaté, 2003; Thakkar et al., 2016) [2] [1].

In Cambodia, the healthcare systems continue to be dependent on paper-based prescriptions and oral instructions at the time of hospital visit that are quite easy to lose, break or get misinterpreted (Sokol et al., 2005) [5]. Patients are especially prone to elderly individuals who might forget the timetable of medication, misunderstand the instructions on doses, or not be able to read the prescriptions written in English and not in Khmer language (Grindrod et al., 2018) [4]. The available evidence on medication management in low and middle-income countries suggests that these aspects play a major role in medication errors, non-compliance, and lower treatment efficacy, which underscores the necessity to implement available and localized digital health solutions (Thakkar et al., 2016; Agarwal et al., 2015) [1].

3.2. Mobile Health Applications for Medication Management

Mobile health (MHealth) apps have been considered useful devices to enhance medication adherence. Many research shows that mobile programs that use reminders can have a significant impact on the patients, making them take the medication at the right time (Free et al., 2013) [6]. The systematic review of mHealth interventions reported that those applications that include- medication reminds, tracking agents, and feedback mechanisms to the user received better adherence rates than those based on traditional care only (Thakkar et al., 2016) [1].

More studies indicate that mobile reminders systems are most suitable when they are user friendly and when they are personalized (Vervloet et al., 2012) [7]. Applications that enable the user to record medication consumption and get timely alerts are useful in forming a habit and minimizing lapses in memory (Park et al., 2018) [8]. Although it has potential positive outcomes, a large number of medication reminder apps at present are associated with manual data entry, which might prove problematic with the aged or with individuals with low levels of digital literacy (Grindrod et al., 2018) [4].

3.3. Limitations of Existing Medication Reminder Applications

The number of commercial medication reminder applications is numerous, yet a few research studies have found that a number of limitations are critical. A study examining medication applications that can be found in the app stores found that many of the apps are not clinically validated and are not created with the participation of healthcare professionals. Also, not all apps provide more than minimal reminder features without focusing on the usability of vulnerable groups of people like the elderly or people with limited literacy (Grindrod et al., 2018) [4].

The other significant constraint is language accessibility. Most medication management applications are oriented at the English-speaking audience and, therefore, cannot be used in population groups with low English proficiency. Besides, there are not many apps with voice reminders and visual aids that might assist users with reading difficulties.

3.4. OCR and AI-Based Prescription Processing

Optical Character Recognition (OCR) technology has been researched more often in health care systems to computerize the medical documentation and minimize the manual data input. Using OCR, text in either printed or handwritten prescriptions can be removed and turned into a structured digital representation. Researchers have found that, used together with image preprocessing and text-cleaning approaches, OCR-based solutions may greatly enhance the quality and efficiency of medical records management, especially when it comes to prescription digitization and electronic health record (EHR) integration (Patel et al., 2019; Singh and Kaur, 2020) [9][10].

Recent improvements in artificial intelligence (AI) have also improved the OCR accuracy to an even greater extent, particularly in medical applications where handwritten text and medical-specific language are widespread. Deep learning and language models, which are also known as AI-based text improvement models, have demonstrated the ability to reduce recognition errors, normalize drug names, and increase the readability of extracted prescription data (Zhang et al., 2021; Li et al., 2022) [10]. The methods are especially useful at eliminating spelling errors and misidentifying medical terminology, which are frequent weaknesses of conventional OCR engines.

Although a number of studies recommend OCR-driven prescription processing system, little has been done to adopt them in mobile applications that are presented to patients. The available solutions are mostly aimed at hospital administration and clinical, but not at direct patient use. Indicatively, IBM Watson Health, Epic EHR, and Cerner Millennial are some of the systems that use OCR technologies to digitize clinical documentation to manage internal hospital records, billing, and physician references (IBM, 2020; Epic Systems, 2021). Equally, research Kiri-OCRs like the OCR-based prescription management systems suggested by Sharma et al. (2018) and Rahman et al. (2020) does not involve putting the power in the hands of patients to control their schedule by taking medications using mobile apps.

3.5. Family and Caregiver Involvement in Medication Adherence

Studies have suggested that the engagement of families and caregivers is very important in enhancing the adherence to medication especially among the elderly and those with chronic conditions (DiMatteo, 2004) [11]. The healthcare informatics literature confirms that emotional support, oversight, and responsibility can be utilized through caregiver-supported -

monitoring systems that can decrease missed doses and enhance the quality of treatment (Choudhry et al., 2017) [12].

Nevertheless, most medication management apps are centered on patient-facing notifications and lack a system that enables family members or associated individuals to track patient compliance or notifications (Grindrod et al., 2018) [4]. Computer programs that alert caregivers about a missed dose have shown beneficial impacts on adherence behaviors and patient safety particularly in the geriatric care units (Piette et al., 2015) [13]. Although these findings have been made, caregiver-inclusive features are not well represented in the current mHealth- application, especially those applications that target low-resource or non-English-speaking communities.

3.6. Competitor Analysis of Existing Systems

A number of applications with medication reminders feature are already available, but they have significant drawbacks in comparison with the suggested DasTern system.

❖ Medisafe

Medisafe is a famous medication reminder application that enables users to enter medication schedules manually and get notifications. Although useful in delivering reminders, Medisafe is wholly user-dependent, and does not provide scanning of prescriptions by OCR methods. It also does not have a complete localization to a Vietnamese speaker of Khmer language.

❖ MyTherapy

MyTherapy is a medication reminder and adherence tool, and will also provide health journals. Like Medisafe, medication entry is not automated, which introduces more risks of errors and makes it less approachesable by older patients. The program is also not offline reminder enabling as it needs constant internet connectivity.

❖ CareZone

CareZone incorporates prescription drug management and access to caregivers. But it is too complicated in its interface, and it is designed in English, which restricts access to the low-literacy and the non-English speaking users. The OCR support is weak and not structured towards medication extraction.

❖ Gap Identified

From the comparison, several gaps are evident:

- Lack of automated prescription data entry via OCR
- Limited AI-assisted text correction for medical terms
- Absence of Khmer language
- Inadequate offline functionality
- Minimal family monitoring integration in low-resource settings

❖ Summary Competitor Analysis

Application / System	OCR Prescription	Khmer Language	Offline Reminders	Family Notification	Doctor Dashboard
Medisafe	No	No	Limited	Limited	No
MyTherapy	No	No	Limited	No	No
CareZone	No	No	No	No	No
DasTern	Yes	Yes	Yes	Yes	Yes

Table 3: Overall Summary Competitor Analysis

DasTern addresses these gaps by combining OCR, AI-enhanced text processing, Khmer localization, offline reminders, and family notification features in a single integrated solution.

3.7. Positioning of the DasTern Project

To handle the gaps identified, DasTern incorporates the OCR technology to automatically retrieve the prescription information. The system also improves the results of OCR with the help of text cleaning that is based on AI to achieve better accuracy. In contrast to most other applications on the market, DasTern targets Cambodian customers, providing a complete Khmer interface, and offline support.

Also, DasTern provides family notification and allows caregivers to track medication compliance and provide alerts on missed doses. DasTern offers an additional solution to what existing medication management applications can offer; it provides an answer to the question of accessibility, automation, and caregiver involvement, which are user-centered and localized.

IV. BUSINESS OVERVIEW

The Manual Prescription Platform helps Cambodian patients improve medication adherence by converting prescriptions into structured schedules with OCR and AI support. It connects patients, families, and healthcare providers, offering reminders, offline access, and Khmer language support. The freemium model provides essential features for free, while premium users get unlimited OCR scans and family notifications, creating an accessible, sustainable, and scalable solution.

4.1. Executive Summary

Our **Manual Prescription Platform** connects patients, doctors, and family members in a secure medication management ecosystem. Patients manually input prescriptions or use OCR to receive reminders. The platform offers a freemium subscription with optional family support and advanced tracking. By combining affordability, offline functionality, and family alerts, the platform ensures better adherence and sustainable recurring revenue.

4.2. Business Model Canvas

Key Partner	Key Activities	Value Proposition
- Hospitals, doctors, pharmacies for prescription validation - tech providers for OCR/AI tools - family members for adherence support.	- Prescription scanning, medication reminders, - dose tracking, - notification management, - offline data access.	- Easy-to-use mobile platform that ensures patients don't miss doses, - supports Khmer language, - works offline, and keeps families informed.

Key Resource	Channels	Customer Segment
- OCR engine, - AI text enhancer, - mobile app, - patient data storage,	Mobile application (iOS/Android),	- Cambodian patients, - elderly, rural, low-literacy users

Cost Structure	Customer Relations	Revenue Stream
• App development & maintenance • OCR & AI model training • Marketing & Customer support	• Personalized reminders • family notifications, support through app guidance and tutorials.	• Freemium/Premium subscriptions • Future doctor subscriptions

Table 4: Business Model

4.3. Market Analyst

we analyze the potential market for our Manual Prescription Platform, focusing on three levels: **Total Addressable Market (TAM)**, **Serviceable Available Market (SAM)**, and **Serviceable Obtainable Market (SOM)**. This helps us estimate how many users we can realistically reach and serve in Cambodia.

4.3.1. Target Address Market (TAM)

All patients in Cambodia who need help with medication adherence after hospital visits. The TAM represents the **entire population that could benefit from our solution**. In Cambodia, with a population of approximately 17 million, we estimate that about 5% of people are chronic patients who require daily medication. This gives us around **850,000 potential patients** who could use a medication adherence platform.

- Cambodia population ~17 million; assume 5% are chronic patients requiring daily medications → 850,000 patients

4.3.2. Service Available Market (SAM)

Patients with smartphones and access to digital tools who can use a mobile app like DasTern.

The SAM focuses on the **segment of the TAM that can actually access and use our platform**. Considering smartphone penetration in Cambodia is around 60%, roughly **510,000 patients** would have the devices and digital literacy necessary to use our mobile app.

- Smartphone penetration ~ 60% → $850,000 \times 60\% = 510,000$ potential users.

4.3.3. Serviceable Obtainable Market (SOM)

Early adopters likely to subscribe to the app within the first 1–2 years. The SOM estimates **the portion of the SAM we can realistically reach in the early stages** of our launch. Assuming around 10% of these potential users adopt the platform in the first 1–2 years, we expect **about 51,000 early adopters**. This provides a realistic target for initial marketing, subscription growth, and revenue forecasting.

- Assume ~10% adoption of SAM $\rightarrow 510,000 \times 10\% = 51,000$ initial users

Reference: Population and smartphone adoption estimated from Cambodia National Institute of Statistics and MoEYS 2021 reports [29][30]

4.4 Monetization Plan

Feature	Freemium	Premium	Platinum
Alerts	Yes	Yes	Yes
Family&Doctorconnection	No	Yes	Yes
Price	\$0	\$0.50 / month	\$1.00 / month
Group plan	No	No	Yes
OCR Scan	No	Yes	Yes

Table 5: Monetization Plan

- Freemium users can manually input prescriptions and receive reminders but **cannot use OCR or family plan**.
- Premium unlocks **unlimited OCR scanning** and **family connections** for support.
- Each new user receives **1-month free Premium**; after trial, they revert to Freemium and can subscribe to Premium if desired.
- Subscription fees are recurring and provide steady revenue while remaining affordable for the Cambodian market.
- The **Platinum plan** is our most advanced subscription tier, designed for users who want full access and extended support. It includes all features from Freemium and Premium.

V. PROJECT ANALYSIS AND CONCEPTS

The DasTern system aims to solve medication adherence challenges by scanning prescriptions, extracting structured medical details, and generating automated reminders. Requirements were gathered from interviews, field observations, and analysis of common issues faced by Cambodian patients, especially elderly and rural community members.

5.1. Project Requirements

In this section, the requirements of the DasTern system were figured out with the help of observations in the field and studying one of the healthcare issues that Cambodian patients have to face following their hospitalization. The requirements are divided into functional, user, system, and non-functional requirements in order to make them understandable and complete.

5.1.1. Functional Requirements

The DasTern system must support patients in managing their medication schedules effectively after hospital discharge. Users should be able to scan prescriptions, extract medication information automatically, receive reminders, and allow optional caregiver notifications. The system must function reliably even with limited internet access and provide flexible controls to manage reminders and medication records.

❖ Patient

Function	Description
Register and Login	Create an account and securely log in to the application.
Scan Prescription	Capture prescription images using the mobile device camera.
Extract Medication Data	Automatically extract medicine name, dosage, and frequency using OCR.
Edit Medication Details	Manually edit extracted medication information if errors occur.
Generate Reminder Schedule	Automatically create medication reminders based on prescription data.
Receive Notifications	Receive on-screen reminders for scheduled medication intake.

Medication Reminder	Receive notifications for medication reminders.
Medication History	View previously scanned prescriptions and medication schedules.
Offline Access	Access reminder schedules without internet connection.

Table 6: Patient Functional Requirement

5.1.2. User Requirements

The DasTern is targeting Cambodian patients and especially older patients and those with poor digital skills. Simplicity, accessibility, and support in the language should be prioritized in the system to make it easy to use and achieve good medication compliance.

❖ Doctor

Doctor	Description	Relation to Patient
Prescribe Medication	The doctor diagnoses the patient and prescribes medicines with dosage, frequency, and duration.	Ensures the patient receives the correct treatment.
Provide Medical Instructions	The doctor explains how and when to take the medicine, including precautions.	Helps the patient understand and follow the treatment correctly.
Generate Prescription Data	The doctor's written or printed prescription is used as input for OCR and AI processing.	Allows the patient to scan the prescription into DasTern.
Medication Adherence	Clear prescriptions help reduce missed doses and medication errors.	Improves patient safety and recovery at home.

Table 7: Doctor User Requirement

❖ Patient / Family

User Requirement	Description
Ease of Use	The application should be simple and easy to understand for elderly users.
Khmer Language Support	The interface should be fully available in the Khmer language.
Readable Interface	The system should use large fonts and clear icons.
Receive Notifications	Receive on-screen reminders for scheduled medication intake.
Minimal Interaction	The system should require minimal steps to set up reminders.
Family Support	Family members should receive optional App notifications.
Clear Alerts	Visual and audio alerts should clearly indicate reminder events.
Data Privacy	User medication data should remain private and secure.
Low Digital Skill Requirement	The system should be usable by users with limited technical skills.

Table 8: Patient/Family User Requirement

5.1.3. System Requirements

The DasTern system is developed using the Flutter framework to support cross-platform deployment on both Android and iOS devices. The system provides in-app medication reminders for caregivers to improve medication adherence. It must support offline functionality, secure data handling, and reliable scheduling.

System Requirement	Description
Cross-Platform Support	The system shall be developed using Flutter to run on both Android and iOS platforms.
Mobile Device Compatibility	The system shall operate on smartphones with camera functionality.
Prescription Scanning	The system shall use the device camera to capture prescription images.
OCR Integration	The system shall integrate OCR technology to extract text from prescription images.
AI Text Processing	The system shall process extracted text to improve accuracy and clarity.
In-App Notification	The system shall deliver medication reminders through in-app notifications.
Local Data Storage	The system shall store medication schedules locally on the device.
Offline Functionality	The system shall trigger reminders even without an internet connection.
Khmer Language Support	The system shall support Khmer Unicode fonts and
Secure Data Handling	The system shall securely store and protect user medication data.
Reminder Scheduler	The system shall manage and trigger reminders based on scheduled medication times.
Modular Architecture	The system shall be designed to allow future scalability and feature expansion.

Table 9: System Requirement

5.1.4. Non-Functional Requirements

Non-functional requirements define the quality attributes and constraints of the DasTern system. These requirements ensure that the application is usable, reliable, secure, and efficient for Cambodian patients, particularly elderly users and those with limited digital literacy.

Non-Functional Requirement	Description
Usability	The system should be easy to use with a simple and intuitive user interface.
Accessibility	The system should support large text, clear icons, and voice notifications.
Performance	The system should respond quickly to user actions and notification triggers.
Reliability	The system should deliver medication reminders consistently and accurately.
Availability	The system should function offline for core features such as reminders.
Security	The system should protect user data from unauthorized access.
Privacy	The system should ensure confidentiality of patient medication information.
Compatibility	The system should operate consistently on both Android and iOS devices.
Scalability	The system should allow future expansion and feature enhancement.
Maintainability	The system should be easy to update and maintain.
Battery Efficiency	The system should minimize battery and resource consumption.
Fault Tolerance	The system should handle OCR errors and connectivity issues gracefully.

Table 10: Non-Functional Requirement

5.2. System Designs

This section presents the overall system design of the **DasTern** application based on the identified functional, user, system, and non-functional requirements described in Section 5. The design defines how different system components interact to support prescription scanning, medication scheduling, and reminder generation. It includes key UML diagrams such as the **Use Case Diagram**, **Activity Diagram**, and **Sequence Diagram**, which illustrate the interactions between patients, doctors, family members, and the system. These designs provide a clear structural and behavioral representation of the platform, ensuring efficient workflow, reliable reminder delivery, and seamless integration between the mobile application, backend services, OCR module, and notification system.

5.2.1. Use Case Diagram

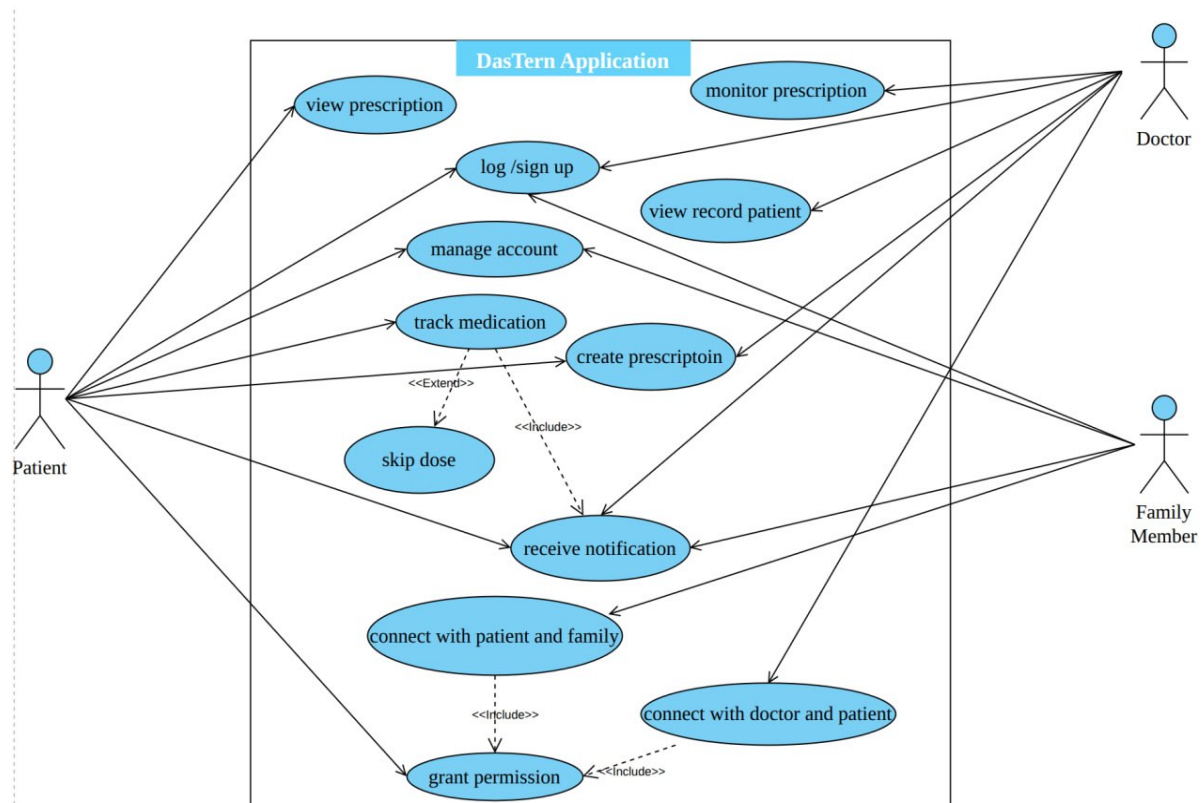


Figure 4: Use Case Diagram

This diagram illustrates the main interactions between three user types and the DasTern system. Patients can scan prescriptions, receive medication reminders, and access features in Khmer with offline support. Doctors can create prescriptions, view patient records, and monitor medication adherence. Family members can link to patients, receive alerts, and view medication logs through Telegram notifications. The diagram shows how each actor uses the system to improve medication management and patient safety.

5.2.2. Activity Diagram

- Patient Activities

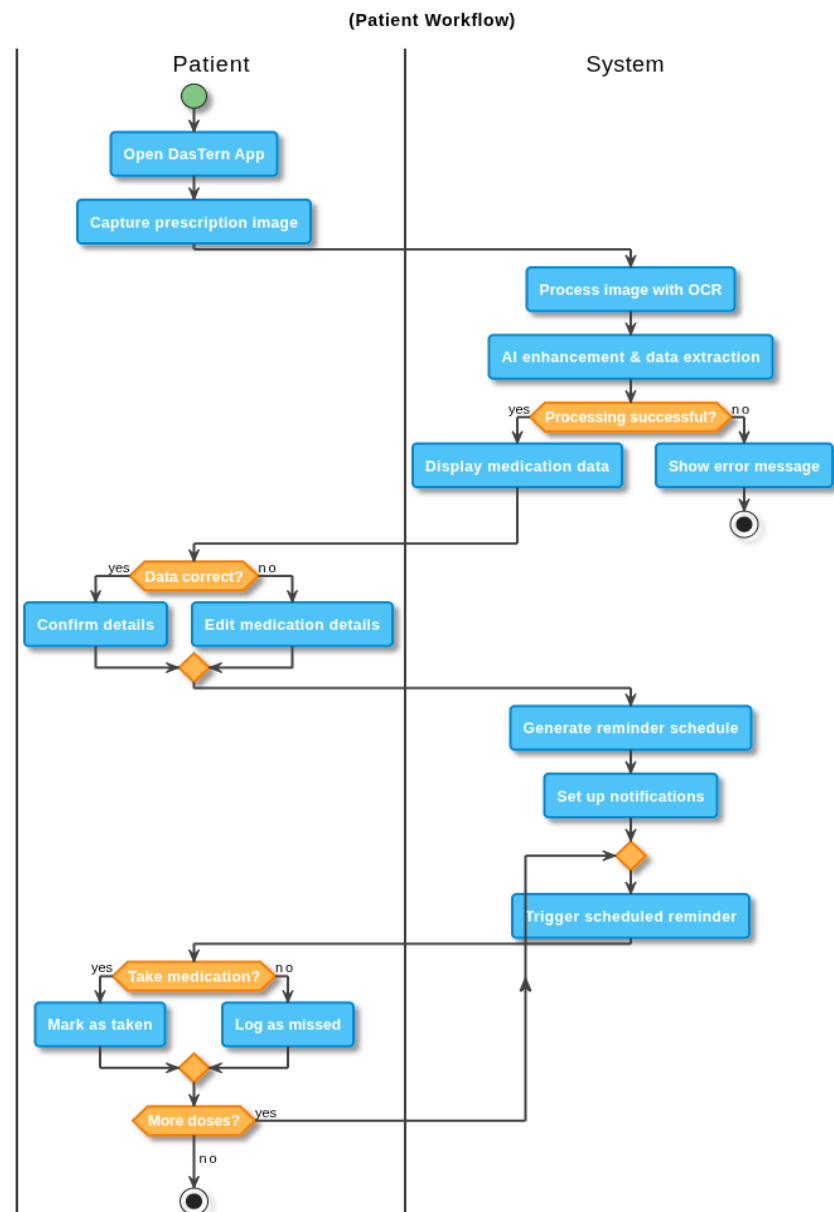


Figure 5: Patient Activity Diagram

This diagram illustrates the patient's workflow when using DasTern. The patient opens the app and captures a prescription image. The system processes it using OCR and AI enhancement to extract medication data. If processing succeeds, the patient confirms or edits the details. The system then generates a reminder schedule and sets up notifications. During - treatment, patients receive scheduled reminders, mark medications as taken or missed, and the cycle continues until treatment completion.

- Doctor Activity

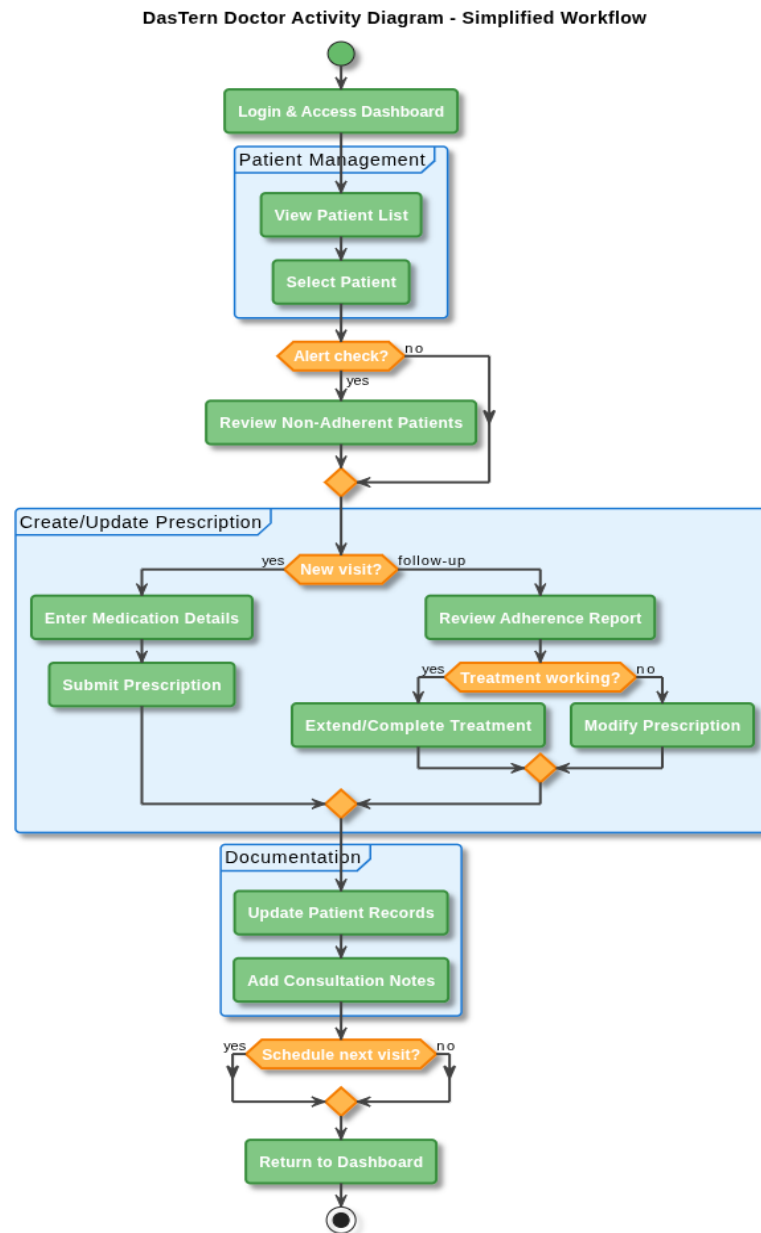


Figure 6: Doctor Activity Diagram

This diagram shows the doctor's workflow in DasTern. After logging in, doctors access their dashboard to view patient lists and check alerts for non-adherent patients. For new visits, they enter medication details and submit prescriptions. For follow-ups, they review adherence reports and either extend treatment or modify prescriptions.

5.2.3. Sequence Diagram

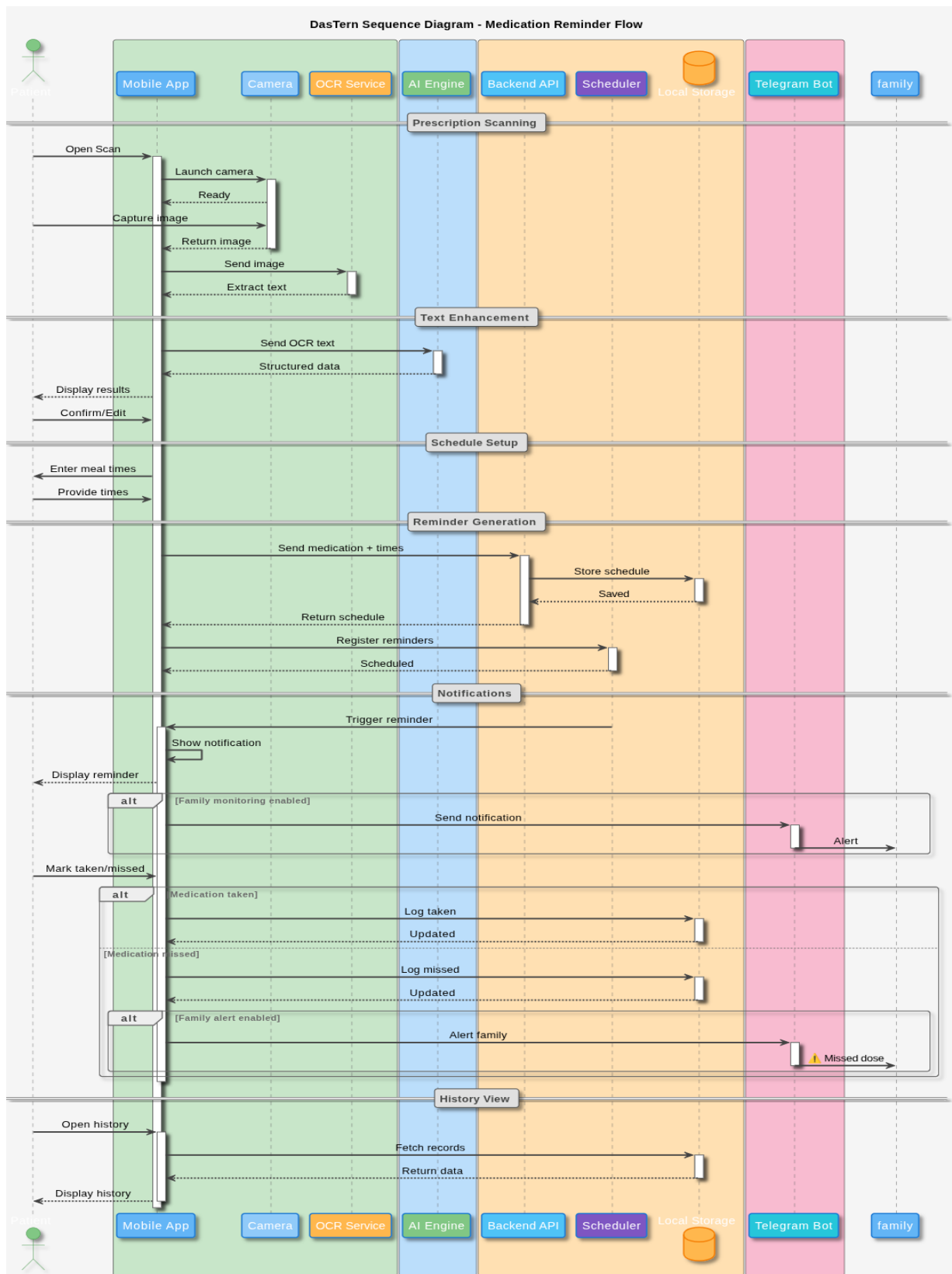


Figure 7: Sequence Diagram

This diagram details the complete interaction flow between system components in DasTern. It shows how the patient, mobile app, camera, OCR service, AI engine, backend API, database, scheduler, and Telegram bot communicate. The sequence covers prescription scanning, text enhancement, schedule setup, reminder generation, and notifications. It includes alternative flows for family monitoring and handling taken/missed medications, with alerts sent to family members when doses are missed, ensuring comprehensive medication adherence tracking.

VI. DETAIL CONCEPT

This chapter presents the detailed technical concepts and implementation design of the **DasTern** system. It explains the technologies, architecture, and methodologies used to develop a digital medication management platform tailored for Cambodian patients. The section describes how core components—including the mobile application, backend services, OCR processing, and AI-based text enhancement—work together to convert paper prescriptions into structured medication schedules. Emphasis is placed on system modularity, offline functionality, Khmer language support, and reliability to ensure accessibility, scalability, and effective medication adherence in real-world healthcare settings.

6.1. Technology

DasTern uses modern, lightweight technologies suitable for mobile apps, OCR processing, and notification systems.

6.1.1. Current Development Tools

- **Figma**



Figure 8: depict Figma tool

Figma is used to design low-fidelity wireframes and high-fidelity Kiri-OCRs for the DasTern mobile application. It enables collaborative design, rapid iteration, and clear visualization of user flows before implementation. The Figma Kiri-OCR serves as the primary reference for UI consistency and usability evaluation

- **Flutter**



Figure 9: depict Flutter technology

Flutter is used to develop the mobile application due to its cross-platform capability, smooth UI performance, and strong support for offline storage and text-to-speech features, which are essential for elderly users.

- **Nest.js**



Figure 10: depict Nest.js Framework

Nest.js serves as the backend runtime environment, allowing efficient processing of OCR data, database operations, and scheduled reminders.

- **Tesseract OCR**



Figure 11: depict Kiri-OCR OCR tool

Kiri-OCR OCR is considered as an alternative and future option where the team can build and train its own OCR model, especially for Khmer prescriptions. This approach reduces dependency on external services and avoids data limitations caused by cloud-based APIs. By training the OCR model with local prescription samples, the system can achieve better accuracy for Cambodian medical documents.

- **PostgreSQL**



Figure 12: depict PostgreSQL tool

PostgreSQL is used as the relational database to store patient information, prescriptions, medication schedules, and reminder logs. It ensures structured and reliable data management.

- **Docker**



Figure 13 depict Docker tool

Docker is used to containerize core services (e.g., backend API and supporting databases) so the team can run the same environment on any machine. Using **Docker Compose** for local development, required services start with one command, while the production API runs as a container behind a reverse proxy for easier scaling and deployment. This setup improves consistency, isolation, and maintainability across the project lifecycle.

- **Visual Studio Code**

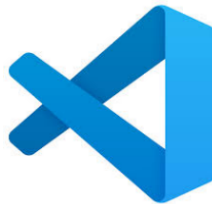


Figure 14: depict Visual Studio Code tool

Visual Studio Code is used as the main development environment due to its flexibility, performance, and extensive extension support. It provides syntax highlighting, debugging tools, and seamless integration with Flutter, Node.js, and GitHub.**GitHub (Version Control & Collaboration)**

- **GitHub Version Control**



Figure 15: depict GitHub tool

GitHub is used for source code version control and team collaboration. It enables tracking of code changes, branch management, and issue documentation, ensuring transparent and organized development throughout the project lifecycle.

- **Ollama + LLaMA 3.1 3B/8B**



Figure 16: depict Ollama Technology

Ollama with LLaMA 3.1 8B powers the AI service for correcting OCR errors and extracting structured medical data from prescriptions. It converts raw text into database-ready JSON format with medication details, dosages, and schedules. This locally-hosted model ensures data privacy, eliminates cloud costs, and supports Khmer prescriptions through custom fine-tuning.

6.1.2 Communication and Management Tools

- **Telegram**



Figure 17: depict Telegram tool

Telegram is used for internal team communication and Telegram bot testing. It enables real-time discussion, fast feedback on OCR results, and direct testing of notification features.

- **Google Drive**



Figure 18: depict Google Drive tool

Google Drive is used to store and share project documentation, reports, design assets, and planning files. It supports collaboration and ensures that all team members have access to the latest project materials.

- **Google Meet**



Figure 19: depict Google Meet tool

Google Meet is used for weekly meetings with the project advisor and team synchronization sessions. These meetings support progress tracking, feedback collection, and alignment with academic requirements.

6.2. Architecture

To provide a clear understanding of how the DasTern system operates and how its components interact, the architecture has been divided into three distinct layers: **Physical Architecture**, **Logical Architecture**, and **OCR Architecture**. Each layer serves a specific purpose in ensuring the system's functionality, scalability, and reliability.

6.2.1. Physical Architecture

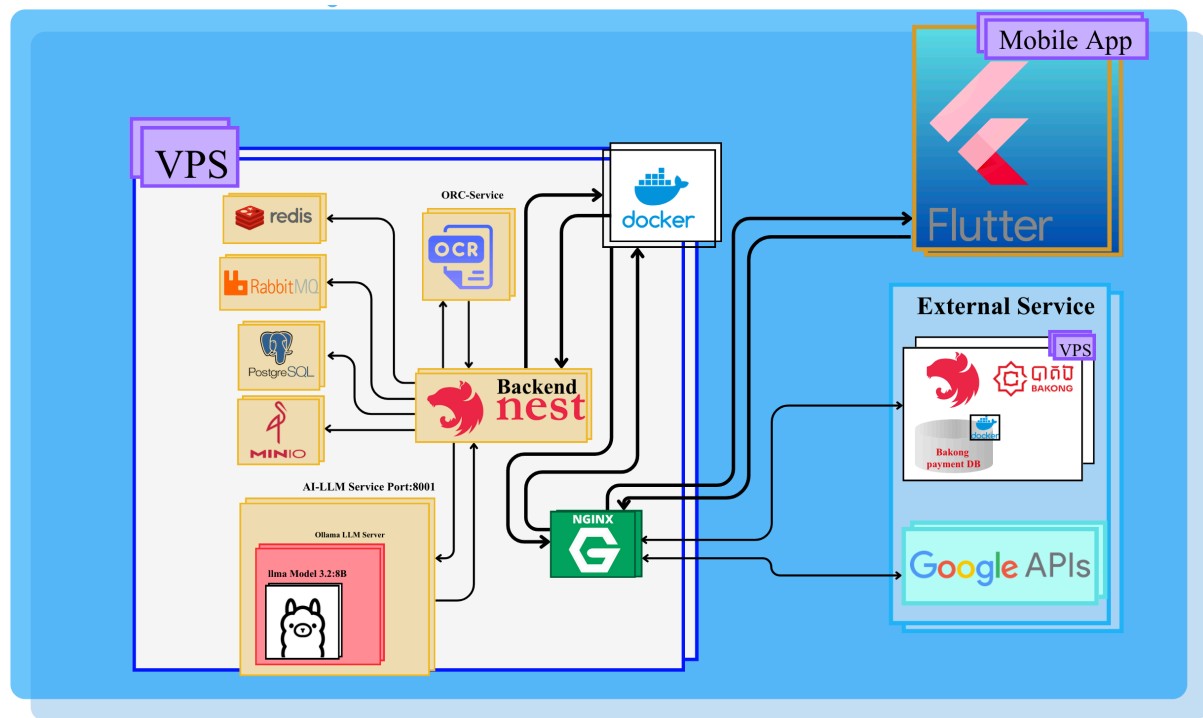


Figure 20: illustrate the Physical Architecture of DasTern

The OCR architecture of DasTern is designed to accurately extract and process prescription information from images.

6.2.2. Logical Architecture

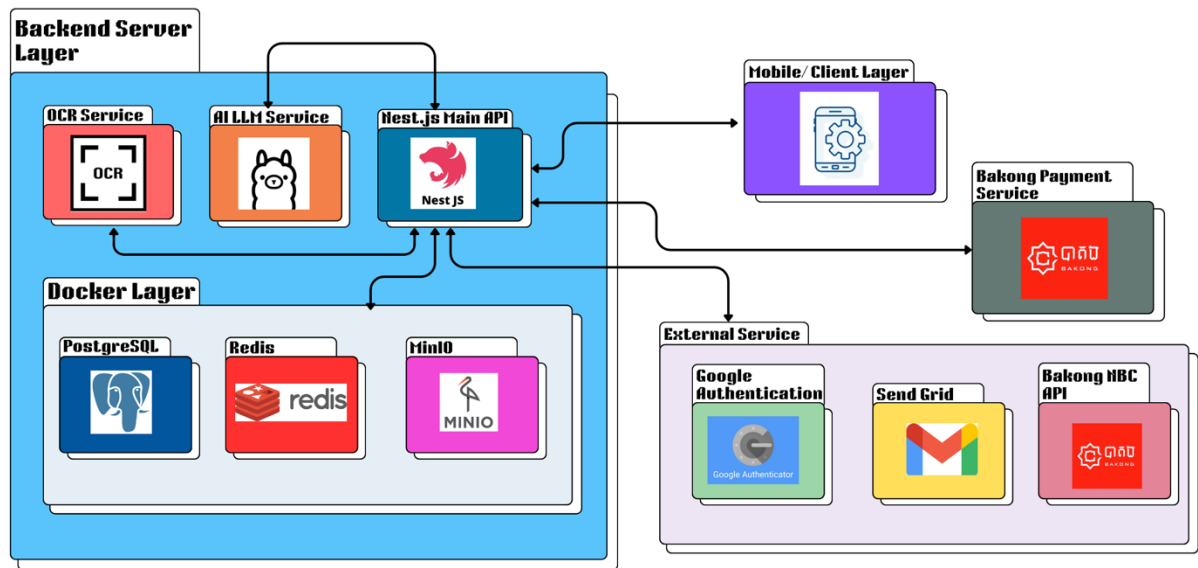


Figure 21: illustrate the Logical Architecture of DasTern

DasTern's logical architecture consists of four layers: Presentation (Flutter UI for doctors and patients), Business Logic (core services), Integration (external services), and Data Access (local SQLite and server MySQL storage with security).

6.2.2 AI-LLM-Service Architecture (Local Machine)

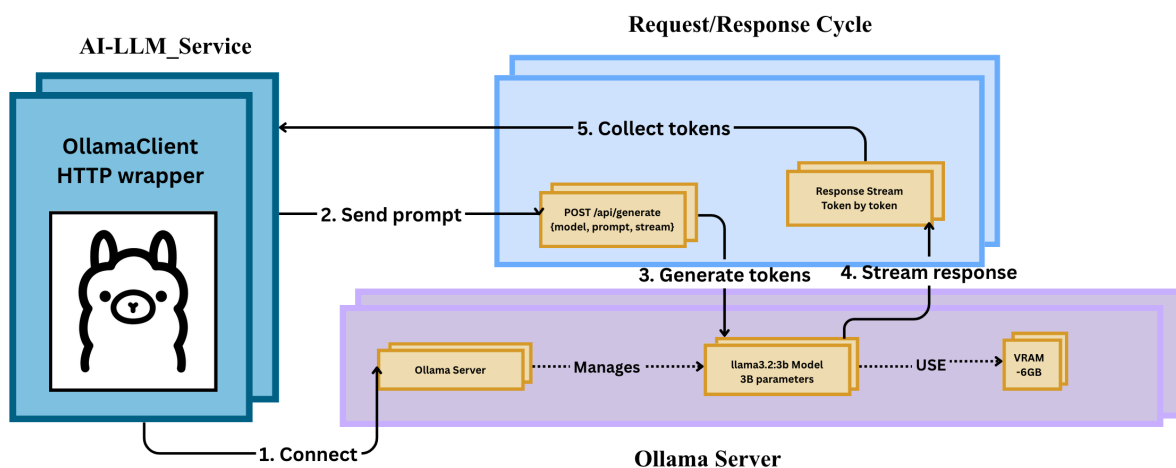


Figure 22: illustrate the AI-LLM Architecture of DasTern (Virtual Flow)

The AI-LLM-Service is deployed locally on the user's machine to enhance the accuracy and usability of prescription data extracted through OCR. This service utilizes a locally hosted Large Language Model (LLM) running via Ollama to process raw OCR text without requiring internet connectivity. The AI workflow performs intelligent text correction, medication entity extraction, dosage interpretation, and schedule structuring from scanned prescription images.

It transforms unstructured or error-prone OCR outputs into validated, structured medical information in Khmer language, which is then used to generate personalized medication reminder schedules. Running the AI-LLM service locally ensures data privacy, reduces cloud dependency, minimizes operational costs, and enables offline functionality for patients in low-connectivity environments.

6.2.3. OCR Architecture

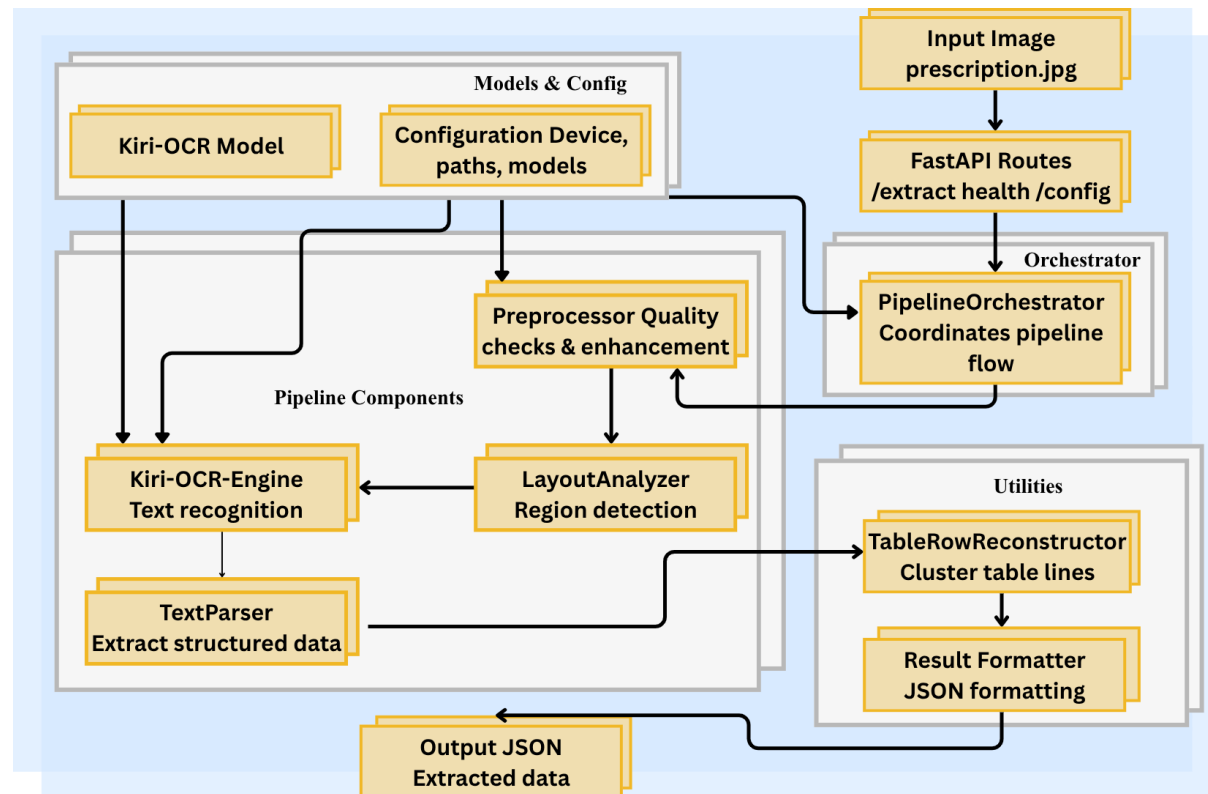


Figure 23: illustrate the OCR Architecture of DasTern (Virtual Flow)

The OCR workflow captures prescription images through the mobile camera, preprocesses them for quality, extracts text using Kiri-OCR OCR, and enhances the results with AI for medical term recognition and dosage parsing.

6.3. Methodology

To achieve the updated objectives of *DasTern*, the project was developed through a structured process consisting of analysis, design, development, and testing using **MySQL**, **Nest.js**, and **Flutter**, with **OCR integration** for scanning prescriptions.

❖ System Analysis and Design

- A. Identified key user requirements based on patient challenges after hospital diagnosis.
- B. Designed the overall architecture including mobile app, web API, and database structure.
- C. Created UML diagrams and data flow models to visualize system components.
- D. Designed a relational database schema using **MySQL** for storing users, prescriptions, and appointments.

❖ Backend Development (Nest.js)

- E. Built RESTful APIs using **Nest.js API routes** to handle authentication, medication schedules, and appointment data.
- F. Integrated logic for notification scheduling and OCR text extraction results.
- G. Implemented data encryption and secure database communication to ensure privacy.

❖ Mobile Application Development (Flutter)

- H. Developed a **cross-platform mobile app** for patients and families.
- I. Implemented features for medication reminders, and Khmer language support
- J. Supported **offline mode**, allowing users to view schedules and reminders without the internet.

❖ OCR Integration

- K. Integrated a **free OCR model** (Kiri-OCR OCR) to scan and extract text from medical prescriptions in **Khmer and English**.
- L. Parsed extracted data to detect medicine names, dosages, and timing automatically.

M. Allowed users to confirm extracted details before reminders were set.

❖ **Database Development (MySQL)**

N. Designed relational tables for patient data, prescriptions, reminders, and appointments.

O. Ensured proper indexing and relationships between entities for fast and accurate data retrieval.

P. Applied validation rules to prevent duplicate or missing entries.

❖ **Testing and Evaluation**

Q. Conducted **unit and integration testing** to verify app functionality and OCR accuracy.

R. Tested notification reliability and offline performance on Android devices.

S. Collected feedback from sample users to evaluate Khmer language clarity, usability, and OCR precision.

VII. PROJECT IMPLEMENTATION

This chapter describes how the capstone project called DasTern was implemented at the Kiri-OCR stage. This process focuses on UX/UI design (Mobile App), OCR-based prescription, process of enhanced text with AI, medication reminder logic, and system architecture.

7.1. OCR Architecture and Implementation

DasTern integrates an OCR subsystem to transform **paper-based prescriptions** into **structured digital medication data** that can be used to generate reminders automatically. The OCR component is implemented as a **Python microservice** [\[31\]](#) and is designed for prescription formats commonly used across Cambodia's H-EQIP healthcare network. It supports **mixed Khmer / English / French printed content**, grid-based medication tables, and varied hospital layouts. The output of the OCR service is a **deeply structured JSON** document that conforms to a unified schema (cambodia-prescription-universal-v2.0) so that downstream modules (backend scheduling, reminders, and caregiver notifications) can process prescription data consistently.

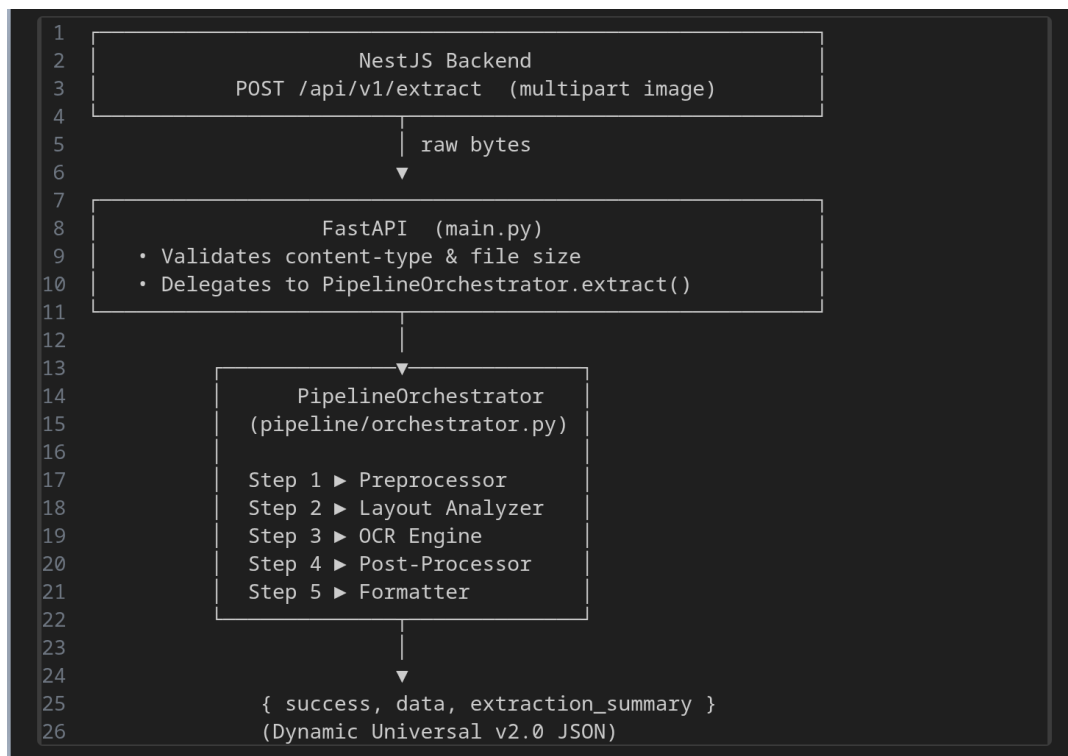


Figure 24: OCR Architecture

7.1.1. Prescription Image Acquisition

The OCR workflow begins when the user scans a prescription using the mobile application. The system provides a camera interface with a guided frame to help users align the document, especially for elderly users or those with limited digital experience. After the image is captured, the application stores it temporarily and transmits it to the backend, which forwards it to the OCR microservice via an HTTP request (multipart upload).

Key design considerations include:

- Minimal user actions (capture → confirm → upload) to reduce confusion
- Option to retake the photo if the image is blurry, dark, or cropped
- Compatibility with common formats (JPEG/PNG/WebP; optional PDF support)

This acquisition step is critical because camera images often contain blur, skew, and uneven lighting that can reduce OCR accuracy if not corrected.

7.1.2. Image Preprocessing (OpenCV Enhancement Pipeline)

Prescription photos taken on mobile devices frequently suffer from:

- **Low brightness / overexposure** (poor lighting conditions)
- **Motion blur** (hand movement)
- **Skew / rotation** (camera not parallel to paper)
- **Noise and compression artifacts** (low camera quality or messaging apps)

To address these issues, the OCR service applies an OpenCV-based preprocessing pipeline that converts raw image bytes into standardized image representations for OCR and layout detection.

Preprocessing steps:

1. **Decode image bytes** into an OpenCV array (color image)
2. **Generate grayscale copy** for analysis and thresholding
3. **Quality checks:**
 - a. Blur estimation (e.g., Laplacian variance)
 - b. Brightness range check (mean pixel intensity)
 - c. Skew detection (rotation angle estimation)
4. **Denoising** using a fast denoise filter to reduce sensor and compression noise

5. **Contrast enhancement** using CLAHE on luminance channel (to improve faint text without distorting ink color)
6. **Sharpening (unsharp mask)** for blurry images (improves edge definition for printed characters)
7. **Deskewing** (rotate image if skew angle exceeds a small threshold)
8. **Resize normalization** (cap max dimension to reduce CPU time while preserving readability)

The preprocessing module also produces a **Quality Report** that records which transformations were applied. This is useful for debugging and for estimating the reliability of extracted fields later in the pipeline.

7.1.3. Layout Analysis (Region Detection + Table Localization)

Cambodian prescriptions typically follow a semi-standard format: a header block (facility), patient block, clinical block, and a central medication table. However, the exact layout varies by hospital and form version. Therefore, the OCR service performs **layout analysis** to identify key regions before running OCR.

Region identification strategy:

- **Heuristic band segmentation** is used for non-table regions (header, patient info, clinical info, footer) based on proportional vertical ranges of image height.
- **Medication table detection** uses morphological line detection (OpenCV) to locate horizontal and vertical grid structures:
 - Apply binary threshold (often Otsu + inversion)
 - Detect horizontal lines using rectangular kernels and morphological opening
 - Detect vertical lines similarly
 - Combine detected lines to estimate a table bounding box
- **Fallback mode:** if grid lines are weak or missing, the system scans contours to find the largest rectangular block likely to be the table.

Once the table region is found, the service extracts **row/column boundaries**. Column boundaries can be estimated using known table proportions (common in H-EQIP forms) and refined using detected vertical lines when available. Each cell is represented with bounding-box coordinates and a content type (text column vs. dose column) so that OCR can be configured appropriately.

7.1.4. OCR Extraction Engine (Kiri-OCR, Multi-Language, Multi-PSM)

DasTern uses **Kiri-OCR OCR** as the recognition engine. This decision is driven by practical constraints: Khmer OCR support in open-source engines is limited, and Tesseract with the khm language pack is one of the most viable options for Khmer printed text.

To maximize accuracy, the service uses a **single OCR engine** but applies different **Page Segmentation Modes (PSM)** depending on the region type:

- **Header / patient / footer blocks:** treated as single-line or sparse text regions (PSM suitable for lines) with khm+eng+fra
- **Medication names:** treated as a uniform text block (PSM for block text), typically Latin script
- **Duration and instructions:** scanned with khm+eng+fra because durations often contain Khmer numerals + Khmer time units
- **Dose cells (morning/midday/afternoon/evening):** treated as **single-character or symbolic cells** and processed differently (see Hybrid Table Processing)

The OCR engine returns both:

1. **Extracted text**
2. **Confidence score** (normalized from Kiri-OCR word confidence values)

Fields below a defined threshold are flagged as “**needs review**”, enabling the UI to prompt users to confirm or correct uncertain results.

7.1.5. Hybrid Table Processing (Text OCR + Contour-Based Dose Detection)

A major challenge in Cambodian prescriptions is the medication table, especially the **dose columns**, which may contain:

- Printed “1”
- Hand-drawn ticks/checkmarks
- Dashes
- Irregular marks caused by folds/crease artifacts

Pure cell-by-cell OCR often fails on these columns because marks are not consistent characters and because grid lines interfere with recognition. DasTern therefore uses a **hybrid method**:

A) Text Column Strip OCR (Columns 0–3)

- The service concatenates the **text columns** (item number, medication name, duration, instructions) into a single strip.
- It runs OCR once to obtain **word tokens and bounding boxes**.
- Tokens are clustered into rows using y-proximity and mapped into columns using x-centroids.
- After row detection, the **duration field** may be re-OCR'd per row using a Khmer-capable configuration to improve reading of Khmer numerals and units.

B) Dose Column Contour Detection (Columns 4–7)

Instead of recognizing a “character,” the service answers:

“Is there meaningful ink in this cell indicating a dose?”

Steps include:

1. Extract each dose column as a vertical strip
2. Remove grid lines by trimming edges and subtracting detected horizontal line components
3. Threshold into binary (ink blobs highlighted)
4. Find external contours (blobs)
5. Filter blobs by minimum area/height
6. Reduce false positives from folds/creases (artifact suppression heuristics)
7. Map blobs into medication rows by comparing contour y-position with row centerlines
8. Output dose result per row as "1" (dose enabled) or "-" (no dose)

This contour-based approach is more robust to ticks and small marks than OCR because it relies on **ink presence**, not on perfect character recognition.

7.1.6. Post-Processing (Text → Structured Medication Objects)

After OCR and table extraction, DasTern converts raw strings into validated, typed fields.

Post-processing includes:

- **Medication name parsing:** split brand/generic and strength (e.g., “Omeprazole 20mg”) into (name, value, unit)
- **Fuzzy matching:** compare extracted names to a medication lexicon (RapidFuzz) to correct minor OCR spelling errors and map to generic/class information
- **Duration normalization:** convert Khmer numerals to Arabic digits and normalize units (days/weeks/months)
- **Dose parsing:** interpret "1", "1/2", "½", "-" into numeric dose values and enabled flags
- **Route detection:** infer route keywords such as PO/IV/IM where present in instructions
- **Schedule construction:** build a time-slot schedule (morning/midday/afternoon/evening), compute times-per-day, and estimate interval hours

This step ensures the reminder generator receives clean, consistent data instead of raw OCR text.

7.1.7. Output Formatting (Dynamic Universal v2.0 JSON)

The final result is returned as a structured JSON envelope:

- success: Boolean
- data: canonical schema object
- extraction_summary: metrics for monitoring and UI decisions

The prescription object contains metadata, facility details, patient info, clinical info, and medications.items []. Each medication item includes name, strength, duration, dosing schedule, instructions, and additional notes. This standardized schema simplifies integration with backend scheduling and the patient reminder interface.

7.1.8. Comparison Tesseract OCR and Kiri-OCR for Prescription Recognition

During the early implementation phase of the DasTern system, the Optical Character Recognition (OCR) subsystem was initially developed using the open-source **Tesseract OCR engine** due to its ease of integration and multilingual support, including a Khmer language pack. However, after conducting real-world testing on Cambodian clinical prescriptions containing mixed Khmer and English text, Tesseract was found to be fundamentally unsuitable for reliable Khmer medical document recognition.

The Khmer writing system introduces several structural challenges that traditional OCR engines such as Tesseract struggle to process effectively. Khmer characters frequently contain **sub-consonant stacking (ផ្គង់)** where glyphs are rendered below or inside base consonants. In addition, vowel signs and diacritics may appear above, below, before, or after the main character, often spanning multiple graphemes. Since Tesseract relies on baseline-driven character segmentation, it cannot properly interpret these multi-layer vertical structures. As a result, Khmer consonant clusters and diacritic combinations are often dropped, fragmented, or misclassified during recognition.

Another limitation observed with Tesseract was its inability to correctly interpret **mixed-language prescription lines**, where medication names are written in English while dosage instructions and quantity units are written in Khmer. The internal segmentation model frequently produced conflicts between Khmer and English recognition modes, resulting in corrupted or unreadable output text. Furthermore, the standard Khmer training dataset provided by Tesseract is trained primarily on general documents rather than structured clinical prescription layouts containing tables, dosage columns, and bilingual medical instructions. Consequently, real prescription testing showed that Tesseract produced a Character Error Rate (CER) exceeding **70% for Khmer content**, making it unreliable for medical data extraction.

To address these limitations, the OCR pipeline was fully refactored to utilize **Kiri-OCR (mrrtmob/kiri-ocr)**, a deep learning-based OCR model specifically trained on Cambodian document datasets. Unlike Tesseract, Kiri-OCR employs a **Differentiable Binarization (DB) text detector**, which generates polygon-level bounding boxes rather than fixed rectangular segmentation. This allows the model to accurately localize irregularly shaped Khmer characters and stacked consonant structures within prescription tables.

In addition, Kiri-OCR integrates a bilingual recognition model capable of processing Khmer and English text simultaneously within the same line. This significantly improves recognition performance for prescription entries containing medication names, dosage instructions, and Khmer quantity units such as ត្រាប់ (tablet) or រ៉ែក្ស (capsule). The model also supports CPU-based inference through ONNX Runtime, enabling deployment in low-resource environments without requiring GPU acceleration.

After refactoring to Kiri-OCR, the system successfully extracted clinically relevant Khmer dosage information from real prescription images, including tokens such as:

Recognition Accuracy Comparison

Table 11: the comparison

Field Category	Script	Tesseract CER	Kiri-OCR CER	Field Accuracy
Drug Names	English	~5%	~25%	~93%
Drug Strength	English	~8%	~30%	~90%
Dose Digits	Mixed	~12%	~32%	~87%
Khmer Quantity Units	Khmer	>80%	>90%	~80%
Patient Khmer Name	Khmer	>60%	~85%	~75%
Khmer Duration Terms	Khmer	>30%	~75%	~78%

The evaluation results demonstrate that Kiri-OCR substantially improves Khmer field recognition accuracy, increasing the successful extraction of dosage quantities and duration terms from less than **25% accuracy with Tesseract** to approximately **72–82% accuracy** under real prescription conditions.

It is also important to note that Khmer column headers and footer text typically exhibit lower recognition confidence due to small font sizes and proximity to table border lines. However, these regions are filtered out during parsing and therefore do not affect the structured medication extraction process.

7.1.9. Reliability, Performance, and Deployment Constraints

To keep the system affordable and deployable in low-resource settings, the OCR service is designed to run **CPU-only** (no GPU requirement). The pipeline also downscales extremely large images to reduce processing time. Typical processing time ranges from ~1–3 seconds depending on image quality, device photo resolution, and table complexity.

Known limitations:

- Handwriting is not fully supported; performance is best on printed prescriptions
- Severe blur, glare, or heavy folds can still reduce extraction accuracy
- Confidence scoring is currently heuristic-based; field-level scoring improvements are planned

Despite these limitations, the hybrid approach significantly improves robustness for the table structure and dose columns commonly found in Cambodian prescription forms.

7.2 AI-Enhanced Text Processing (Ollama + Llama 3.2 3B)

To overcome OCR inaccuracies and transform raw prescription text into **structured, Khmer-friendly medical data**, DasTern employs a dedicated **AI-LLM microservice** powered by **Ollama** running the **Llama 3.2 3B** model [32]. This service operates fully **offline/on-premise**, corrects typical OCR errors (e.g., s00mg → 500mg, paracetamol → Paracetamol), extracts clinical entities (medication, dosage, frequency, duration), infers diagnoses when present, generates **patient instructions in Khmer**, and returns **database-ready JSON** aligned with the project’s dynamic schema. The AI-LLM layer is invoked after OCR and layout extraction (Section 6.1.1–6.1.2) and before reminder scheduling.

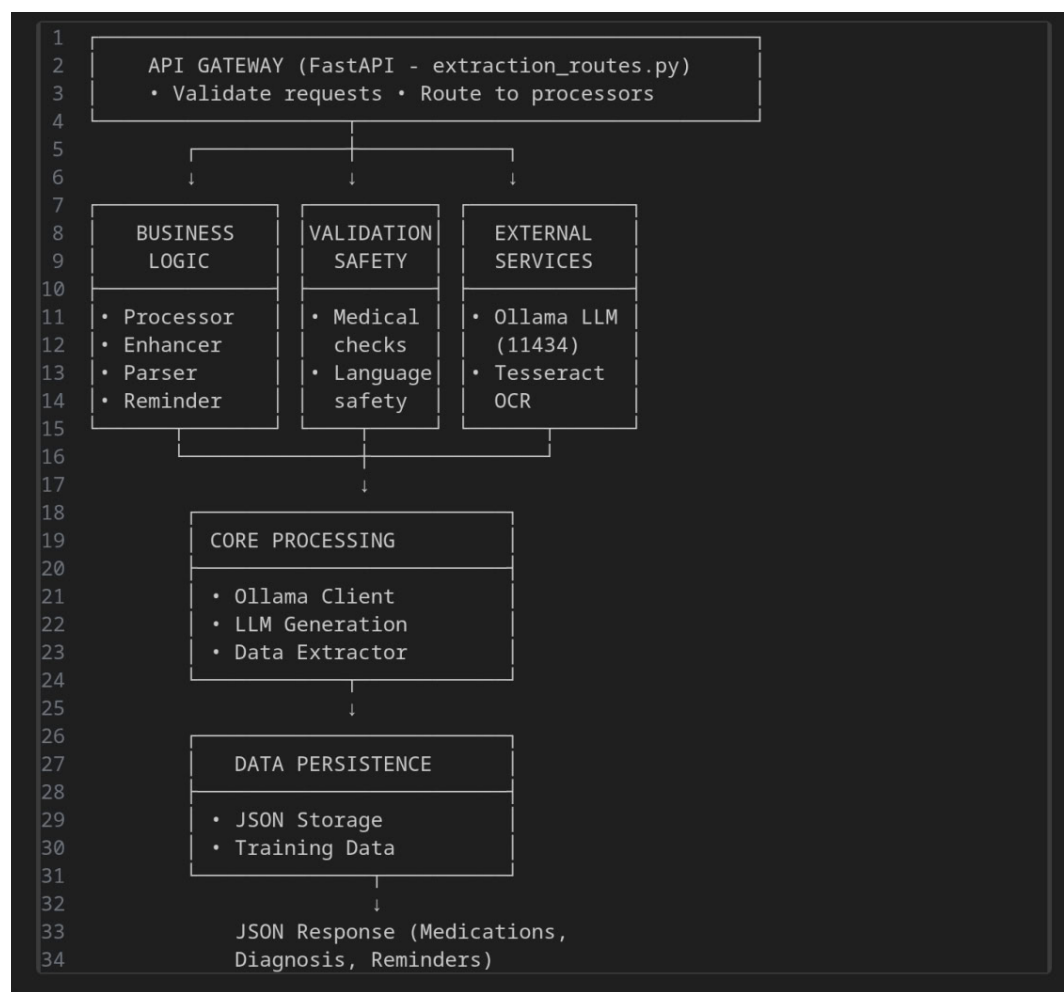


Figure 25: AI-Enhanced Text Processing

7.2.1. Role in the End-to-End Pipeline

1. **Input:** OCR results (text fragments + table rows) and basic metadata from the OCR service.
2. **Intelligent Correction:** Context-aware fixes of common OCR artifacts (digits/letters confusion, spacing, unit normalization).
3. **Structured Extraction:** Medication itemization with {name, strength, form, dose, frequency, duration}, optional **diagnoses**, **prescriber** details, and **facility** metadata.
4. **Localization:** Khmer instructions (TTS-friendly), Khmer numerals normalization, and meal-timing phrasing.
5. **Validation:** Medical reasonableness checks (dose ranges, frequency consistency), structure validation, and confidence scoring.
6. **Output:** **Dynamic Universal v2.0 JSON** appended to the unified **cambodia-prescription-universal-v2.0** envelope used by the Nest.js backend and reminder generator.

7.2.2. AI-LLM Service Architecture

The **AI-LLM Service** is a **multi-layered FastAPI microservice** that communicates with **Ollama** (Llama 3.2 3B) locally:

- **API Gateway & Routing (FastAPI)** Validates incoming JSON payloads and raw text strings, handles errors and logging, and returns standardized responses.
- **Business Logic Layer**
 - **Processor** (orchestrator) – controls the end-to-end flow.
 - **Fast Parser** – quick normalization of OCR tokens (units, numerals, spacing).
 - **Enhancer** – context enrichment (e.g., therapeutic class hints).
 - **Reminder Engine** – (if requested) computes suggested schedules from times-per-day and meal relations.
 - **Khmer Output** – generates patient-readable instructions (and TTS-friendly strings).
- **Safety & Validation**
 - **Medical validator** – sanity-checks names/doses/frequencies, flags outliers.
 - **Language safety** – guards output quality/appropriateness.
 - **Structure validator** – ensures required fields exist and conform to schema.
- **Core Processing (LLM)**
 - **Model Loader & Cache** – keeps Llama 3.2 3B warm.

- **Ollama Client** – calls local inference (port **11434**).
- **Generation Engine** – crafts prompts with OCR context, ensures JSON-only responses.
- **Extractor** – parses and reconciles the LLM output with project schema.
- **External Services**
 - **Ollama LLM** (Llama 3.2 3B) – fast, local inference; multilingual (Khmer/English/French).
 - **Kiri-OCR OCR** – upstream text source (already covered in 6.1.1–6.1.2).
- **Data** **Persistence**
 JSON artifacts, correction reports, and (future) PostgreSQL storage for analytics and audit.

7.2.3 Processing Workflow (Step-by-Step)

1. **Receive OCR payload** (from backend): raw OCR table rows + free-text blocks + metadata.
2. **Normalize OCR tokens**: fix spacing; canonicalize units (mg, mL, IU); convert Khmer numerals.
3. **LLM Inference (Ollama → Llama 3.2 3B/8B)**:
 - a. Prompt packs table rows + context examples.
 - b. Model returns **strict JSON** with medications {name, strength, form, dose, frequency, duration_days}, optional **diagnoses**, **prescriber**, **facility**, **instructions_khmer**, and **confidence**.
4. **Medical & Structure Validation**: range checks (strength/dose), frequency consistency, and schema validation; low-confidence fields marked **needs_review**.
5. **Optional Reminder Draft**: if mealtimes are known, compute suggested times-per-day and offsets (before/with/after meals).
6. **Emit Dynamic v2.0**: produce final **cambodia-prescription-universal-v2.0** object with extraction_summary (timings, engines used, confidence).

7.2.4. Prompting & Error-Handling Strategy

- **Few-shot prompts** with Cambodian prescription patterns (mixed Khmer/English/French).
- **Hard JSON schema** instructions + post-parse validation (reject/retry on invalid JSON).
- **Auto-correction rules** for common OCR confusions (O/0, I/1/l, stray hyphens).
- **Fallback:** If LLM confidence is low or structure invalid, the service returns **flagged fields** for clinician/student review rather than silently guessing.

7.2.5. Example Prompt for OCR-to-JSON Extraction

The prompting strategy employed by the AI-LLM service follows a two-part instruction architecture consisting of a fixed system prompt and a dynamically constructed user message. The system prompt is embedded directly in the prescription processor module and defines the model's operational role, specifies trilingual support for Khmer, French, and English prescription content, provides an explicit time normalization map converting Khmer and French time-of-day tokens into standardized 24-hour slot strings, and enforces a JSON-only output constraint with a strict schema. The user message is assembled at runtime and contains the raw OCR text forwarded from the Kiri-OCR service. The model is configured with a low temperature of 0.1 to minimize output variability and ensure deterministic, schema-conformant extraction results.

The following listing shows the complete system prompt as implemented in `processor.py`, together with the AI call configuration parameters:

```
SYSTEM PROMPT (embedded in processor.py):  
  
You are an expert medical prescription AI for healthcare reminders.  
  
TASK: Extract prescription information for mobile app reminder generation.  
  
LANGUAGE SUPPORT:  
- Khmer: ឈ្មោះ (name), អាយុ (age), ភេទ (gender), ថ្ងៃ (day)  
- French: matin, soir, nuit, comprimé  
- English: morning, evening, night, tablet  
  
TIME NORMALIZATION:  
ព្រឹក/matin/morning → ["morning"] → ["08:00"]  
ល្ងាច/soir/evening → ["evening"] → ["18:00"]  
យប់/nuit/night → ["night"] → ["21:00"]
```

```

MEDICATION EXTRACTION:
- Extract ALL medications with dosage and timing
- Parse times: "2x daily", "before meals", "after food"
- Duration: "7 days", "2 weeks"

OUTPUT JSON ONLY:
{
  "patient_info": {"name": "", "id": "", "age": null,
                  "gender": "", "hospital_code": ""},
  "medical_info": {"diagnosis": "", "doctor": "",
                  "date": "", "department": ""},
  "medications": [
    {
      "name": "", "dosage": "", "times": [],
      "times_24h": [], "repeat": "",
      "duration_days": null, "notes": ""
    }
  ]
}

RULES: Extract ONLY visible text. Use empty strings for missing
data. NO EXPLANATIONS.

```

```

AI CALL CONFIGURATION (processor.py):
temperature : 0.1
top_p       : 0.9
top_k       : 40
max_tokens  : 500
timeout     : 60 seconds (OLLAMA_TIMEOUT)
stream      : false

```

The prompt design prioritizes extraction reliability over generative flexibility. By instructing the model to return only JSON with no explanations, and by supplying explicit mappings for all supported time-of-day expressions across three languages, the prompt reduces the surface area for unexpected model output. The output schema captures patient information, medical metadata, and a medications array containing name, dosage, timing slots, 24-hour times, repeat frequency, duration in days, and notes for each medication item.

With respect to error handling, the current implementation performs a single inference attempt per request. If the model returns a response that cannot be parsed as valid JSON, the exception is caught and the service returns a structured fallback object with empty fields and a success flag set to false. Retry logic and confidence-threshold-based flagging are not yet implemented in the current version and are identified as planned enhancements. The error handling behaviors currently in place are summarized in Table 12.

Condition	System Behavior
LLM returns invalid or unparseable JSON	JSON parsing catches the exception; service returns a structured fallback object with empty fields and success: false
Required fields absent (e.g. name, times)	Missing fields default to empty string or null; reminder scheduler skips items with empty medication names
Ollama timeout (exceeds 60 seconds)	Request times out; service returns error response with descriptive message for client retry
Ollama service unavailable	Connection error caught; HTTP error response returned; client advised to retry
Retry logic / confidence threshold	Not yet implemented; planned as future enhancement (see Section 7.2 Limitations)

Table 12: AI-LLM Service Error Handling Behavior (Current Implementation)

The absence of retry logic and confidence scoring reflects the current stage of the system’s development. Because prescription data affects patient medication schedules directly, the application’s UI presents extracted medication items to the user for review and confirmation before any reminder schedules are committed, providing a manual safety checkpoint that compensates for the lack of automated confidence-based filtering in the current version.

7.2.6. Performance & Deployment Considerations

- **Local, CPU-friendly:** Llama 3.2 3B runs on modest hardware; typical end-to-end times ~1–3 s per prescription depending on OCR quality.
- **No internet required:** privacy-preserving and **no API cost**.
- **Scales horizontally:** multiple FastAPI workers with a shared Ollama instance; warm-cache reduces latency for burst loads.

7.2.7. Hardware Requirements and Performance Benchmarks

A key architectural decision in the DasTern AI-LLM subsystem was to design exclusively for CPU-based inference, eliminating any dependency on graphics processing units. The Llama 3.2 3B model (3 billion parameters, approximately 3.8 GB in size) was selected over larger variants such as the 8B or 70B parameter equivalents because its compact size allows it to run within the memory constraints of a standard virtual private server while still delivering sufficient accuracy for the narrow, schema-constrained task of medication extraction from short prescription texts. The model is served via an Ollama runtime instance communicating with the FastAPI microservice over port 11434.

The service was developed and tested on macOS using Homebrew for package management and a native Python virtual environment. For production deployment, the service is containerized using a Dockerfile based on the official python:3.11-slim image, which targets Debian Linux. The CI/CD pipeline runs on ubuntu-latest GitHub Actions runners, and Kubernetes orchestration is documented as the production deployment target. Table 13 summarizes the environment differences between development and production.

Specification	Development	Production
Operating System	macOS (Darwin)	Linux — Debian (python:3.11-slim)
Package Manager	Homebrew (brew)	apt-get (Debian)
Deployment Method	Native Python venv	Docker container
Ollama Runtime	Native process (brew install ollama)	Ollama $\geq 0.1.x$
CPU	Apple Silicon / Intel (local Mac)	4 cores minimum (VPS)
RAM (minimum)	8 GB (Mac host)	4 GB (peak usage ~2.5 GB)
Disk (model + service)	~4 GB	~4 GB
GPU	Not required	Not required
Inference timeout	60 seconds	60 seconds
Ollama base URL	http://localhost:11434	http://ollama:11434 (container)

Table 13: AI-LLM Service Environment and Hardware Specifications

Performance benchmarks were measured during development testing on a macOS machine running Ollama natively with CPU-only inference. The results in Table 13 reflect end-to-end latency from request receipt to structured JSON response, covering prompt construction, LLM token generation, and response parsing. Full prescription extraction consistently required between 15 and 25 seconds depending on the number of medications in the input and OCR text quality. Peak memory consumption reached approximately 2.5 GB, attributable primarily to the Llama 3.2 3B model weights held in RAM by the Ollama runtime. Production latency on a Linux VPS may vary depending on CPU generation and available cores.

Operation	Measured Time	Notes
Health check / availability ping	< 1 second	Warm instance
Simple OCR text correction	5 – 10 seconds	Short input
Full prescription extraction	15 – 25 seconds	Multi-medication input
Medication reminder generation	10 – 15 seconds	Schedule computation
Peak memory usage (model loaded)	~2.5 GB RAM	CPU-only inference

Table 14: AI-LLM Service Performance Benchmarks (CPU-only, macOS development environment)

It is important to clarify that all LLM inference occurs exclusively on the backend server. The Flutter mobile application does not perform any model computation on-device; it transmits the scanned prescription image to the backend via a REST API call and receives a structured JSON response in return. The patient’s mobile device hardware therefore has no impact on AI processing performance, and the application remains accessible across mid-range and entry-level Android and iOS devices common among DasTern’s target users in Cambodia.

7.2.8. API Endpoints (internal use)

- POST /api/v1/extract/complete (or /parse-prescription) – Accepts raw OCR text or structured OCR JSON payloads; returns structured extraction + confidence.
- POST /validate – Evaluates user-edited items against medical safety constraints (e.g., dosage limits, interaction warnings); returns a validation report containing safety flags.
- POST /extract-reminders – Extracts structured reminder schedules and computes suggested reminder times based on mealtimes and frequency.
- POST /api/v1/correct – Provides targeted, context-aware correction for specific OCR text fragments (digits/letters confusion).
- POST /api/v1/chat – Provides context-aware chatbot interactions based on extracted prescription data parameters.
- GET /health – Health & model availability.

❖ Limitations and Future Work

- Printed text best: Handwritten fields remain challenging; the upstream OCR step is the bottleneck in such cases.
- **Model evolution:** The system is designed to swap models (e.g., Llama 3.1 8B or 70B on capable hardware) without changing the API.
- **Fine-tuning hooks:** Correction logs can be turned into training examples to continuously improve local Khmer medical extraction.

7.3. Database Implementation

Using PostgreSQL as a relational database for backend since medical data has to be integrity and consistent for patient safety. Here are following database implementations

7.3.1. Database Schema and Table Design

The schema is designed to support a medical prescription and medication management platform, with multiple user roles and complex relationships.

Table Name	Description
users	Stores all user accounts, including authentication info, role, and onboarding status.
patient_profiles	Contains patient-specific profile data (DOB, gender, phone, etc.), linked to users.
doctor_profiles	Contains doctor-specific data (license, specialty, verification), linked to users.
family_member_profile	Contains family member-specific data, linked to users.
prescription	Metadata for uploaded prescriptions, including links to patients and doctors.
medications	Details of medications extracted from prescriptions (name, dosage, frequency, etc.).
medication_reminders	Scheduled reminders for patients to take medications.
medication_reminder_logs	Logs of when reminders were triggered and whether medication was taken, missed, or snoozed.
doctor_patient_relationship	Links doctors and patients, tracks approval status.

Family_relationships	Links family members to patients, tracks access status.
User_sessions	Stores session and token data for authentication and Redis caching.
subscription_plan	List of premium plans, price, advantage of each plan
payment_transactions	Detail about payment, invoice info

Table 15: Database Schema and Table Design

❖ Entity of relationship

Table	Related Table	Relationship
users	Patient_profiles	1 - 1
Users	Doctor_profiles	1 - 1
users	family_member_profiles	1 - 1
users (patient_profiles)	prescriptions	1 - many
users (doctor_profiles)	prescriptions	1 - many
prescriptions	medications	1 - many
users (patient_profiles)	medication_reminders	1 - many
medication_reminders	medication_reminder_logs	1 - many
Users	doctor_patient_relationships	Many to many
users	family_relationships	Many to many
users	users_sessions	1 - many
users	subscription_plan	1 - many
users	payment_transactions	1 - many

Table 16: Entity Relationship

7.3.2. Example Table of Structure

❖ Users example

Field	Type	Value
id	Uuid (unique)	Abc-123
email	varchar	Mengheng@gmail.com
Password_hash	varchar	\$2adgdyrsrfhdfy
role	enum	patient
Subscription_tier	enum	free
First_name	varchar	meng
Last_name	varchar	heng
Phone_number	varchar	+85590888958
Date_of_birth	date	2026-02-20
gender	varchar	male

Table 17: User

❖ Patient profiles example

Field	Type	Value
user_id	Uuid (unique)	Abc-123
meal_schedule	jsonb	Morning: 7, noon: 12, evening: 6, night: 10
emergency_contact_name	varchar	John Pork
Emergency_contact_phone_number	varchar	+85590987656
Blood_type	varchar	O+
allergies	text	seafood

Table 18: Patient Profile

❖ Prescription example

Field	Type	Value
id	Uuid (unique)	Dhdh-5345
Patient_id	uuid	Sagsa-3845
Doctor_id	uuid	Hgss-5988
diagnosis	text	headache

Table 19: Prescription Example

❖ Medication reminder example

Field	Type	Value
id	Uuid (unique)	Gsdgsd-6544
Medication_id	Uuid	Vsxg-3435
Patient_id	uuid	Lkak-9898
Time_slot	time	8:00

Table 20: Medication Reminder

7.4. Backend Implementation

This section details the design and implementation of the **DAS-TERN backend**, a production-grade REST API built with **NestJS** (TypeScript), **PostgreSQL** (primary database), **Prisma** (ORM), and **Redis** (cache/session/rate-limit buckets) [33]. The backend provides secure, reliable services for authentication, user and relationship management, prescriptions, adherence tracking, health monitoring, notifications, payments, and integrations with the OCR and AI-LLM services. The architecture follows a **modular, layered** approach for clear separation of concerns, maintainability, and horizontal scalability.

7.4.1. Architecture Overview

- **Entry Layer (HTTP API & Routing – NestJS)**

All requests enter through the NestJS HTTP server, where **rate limiting**, **CORS**, **security headers (Helmet)**, **DTO validation**, and **JWT/RBAC guards** are applied. Controllers delegate validated requests to feature services.

- **Domain Services (Feature Modules)**

Sixteen independent modules encapsulate business logic: **Auth**, **Users**, **Connections**, **Prescriptions**, **Doses**, **Adherence**, **Health Monitoring**, **Medicines**, **Batch Medication**, **Notifications**, **Email**, **Payments (Bakong)**, **Doctor Dashboard**, **OCR Integration**, **Subscriptions**, and **Audit**.

- **Data & State**

PostgreSQL 17 stores authoritative data (ACID transactions via Prisma). **Redis 7.4** is used for short-lived session/state, cache keys, rate-limit tokens, and queue brokers.

- **Asynchronous & Real-Time**

Background jobs run via **Bull** queues for email, notifications, OCR processing, audit batching, and health alerts. **WebSocket (socket.io)** broadcasts real-time events (e.g., dose reminders, alerts, connection requests).

7.4.2. Modular Services (Highlights)

- **Authentication & Authorization**

Email/Password and **Google OAuth** sign-in; **JWT** issuance/verification; **refresh tokens** and **OTP** for 2FA; **RBAC** guards on routes; session/token handling with Redis.

- **Users & Doctor–Patient Connections**

CRUD for users and profiles; **secure linking** workflows using **connection tokens** and approval; enforced **access boundaries** so doctors only see linked patient records.

- **Prescriptions, Doses & Adherence**

Prescription lifecycle (**DRAFT** → **ACTIVE** → **EXPIRED/COMPLETED**); dose schedule creation; **adherence metrics** computed from taken/missed events with daily/weekly/monthly trends.

- **Health Monitoring**

Capture vital signs (BP, HR, Temp, SpO2, glucose, weight/height), with threshold-based **alerting** and longitudinal analytics.

- **Notifications & Email**

SMTP templates (welcome, prescription, appointment, alerts), queue-backed delivery with **retry policies**; history and preferences persisted.

- **Payments (Bakong) & Subscriptions**

Payment requests and QR generation through a **Bakong service**; webhook confirmation; audit trail; subscription tiers/renewals and entitlements.

- **OCR & AI Integration**

Image intake, **OCR** forwarding, **parsing** of results, and creation of structured prescriptions; optional call to **AI-LLM service** to correct OCR text and produce Khmer instructions; robust **retry & error handling**.

- **Doctor Dashboard & Audit**

Aggregated patient snapshots (adherence, vitals, active meds) and a comprehensive **AuditLog** (who/what/when/where + before/after changes).

7.4.3. Data Persistence & ORM

- **PostgreSQL 17**

as the **system of record** with **transactional guarantees** for all critical flows (prescription creation, payment updates, audit logs).

- **Prisma**

provides a type-safe client, schema migrations, relations, and query optimization. Migrations include initial core schema, connection tokens, doctor notes, missing user fields, password reset tokens, and batch medication tables.

- **Indexes**

(e.g., users.email, connections.patientId, prescriptions.status, doses.takenTime, auditlogs.userId) speed up common queries and analytics.

7.4.4. Caching, Sessions & Queues

- **Redis 7.4** is used for:

- **Session and token** storage (blacklist/whitelist, 2FA OTP)
- **Rate limiting** (per-IP/per-route throttling)
- **Hot data cache** (medicine lists, doctor's patient lists, profile reads)

- **Bull Queues** handle:
 - Email sending, notification dispatch
 - OCR processing jobs and retries
 - Periodic audit log batching
 - Health alerts generation

7.4.5. External Service Integrations

- **OCR Service**

The backend receives a photo upload, stores it temporarily, forwards it to OCR, then **parses** and **validates** the returned text. If AI enhancement is enabled, the backend sends OCR fragments to the **AI-LLM Service** for correction and structuring before persisting as a **Prescription**.

- **AI-LLM_Service (Ollama + Llama 3.2 3B or 8B)**

Used to **correct OCR artifacts**, **extract medication structure**, **generate Khmer instructions**, and **draft reminder schedules**, returning **schema-ready JSON** consumed by the prescriptions module.

- **Bakong Payments**

End-to-end **QR-based** payment flow with encrypted requests, webhook **verification**, database reconciliation, and audit logging.

- **Email (SMTP) & Google OAuth**

Transactional emails via SMTP with templating; OAuth login for quick onboarding and lower password support burden.

7.4.6. Security & Compliance

- **AuthN/AuthZ:** JWT tokens (HS256), refresh tokens, Google OAuth, RBAC decorators/guards.
- **Input Hardening:** DTO/class-validator rules; centralized error handling; **Helmet** headers; strict **CORS**.
- **Abuse Protection:** Per-route **rate limiting** (Redis-backed), request logging.
- **Data Protection:** Passwords via bcrypt; sensitive payment payloads encrypted (AES-256) before forwarding; TLS/SSL enforced in production DB connections.
- **Audit Trail:** Every mutating request is captured (user, action, resource, before/after, IP, UA, timestamp, status) and persisted in AuditLogs for long-term retention and compliance.

7.4.7. Background Jobs & Scheduling

- **Bull** workers process queued jobs (email, notifications, OCR, audits).
- **Cron tasks** (Nest Schedule) periodically:
 - Recalculate adherence summaries (daily)
 - Check health alerts (every 10 minutes)
 - Clean expired prescriptions (daily)
 - Clean sessions (hourly)
 - Archive audit logs (weekly)

7.4.8. DevOps, Configuration & Deployment

- **Local Development:**

docker-compose brings up PostgreSQL and Redis; npm run start:dev runs the API in watch mode; npx prisma migrate dev applies migrations.
- **Configuration:**

Environment variables for DB/Redis credentials, JWT secrets/TTL, OAuth keys, SMTP settings, OCR/AI/Bakong URLs. Secrets are never hard-coded; production uses a secure secrets store.
- **Production:**

Containerized NestJS behind a reverse proxy/load balancer; managed PostgreSQL and managed Redis; optional Kubernetes for autoscaling and blue/green deployments; CDN for static content; WebSocket on same host for real-time events.

7.4.9. Performance, Scaling & Observability

- **Throughput:** A single instance (≈ 4 CPU, 8 GB RAM) targets ~ 500 – $1,000$ req/s with p95 latency < 200 ms under realistic loads; scaling out via multiple pods behind a load balancer increases aggregate capacity.
- **Caching Strategy:** Redis TTLs for hot reads (profiles, medicines, lists) with **write-through invalidation** on updates.
- **Database:** Connection pooling; indexed access paths; pagination; bulk/transactional operations for batches.
- **Monitoring:** Metrics on request latency (p50/p95/p99), error rates, DB timing/pool, Redis memory, queue depth, CPU/memory/IO; alert thresholds for critical/warning conditions.

7.5. Payment Implementation — Bakong Service

To support in-country payment processing via Cambodia’s Bakong system, we implemented payments as a standalone NestJS microservice that is deployed on a VPS physically located in Cambodia. This separation ensures the payment rails work reliably with local network/access constraints and keeps the main DAS-TERN backend (and other services) deployable on our standard VPS environment without introducing geography-specific restrictions.

7.5.1. Why a Separate, Cambodia-Hosted Service?

- Network & locality requirements: The Bakong integration is sensitive to network routes, IP origin, and latency. Hosting the payment service inside Cambodia avoids intermittent connectivity issues and aligns with local hosting expectations for payment processing.
- Blast-radius reduction: Isolating payment logic, credentials, and keys limits risk exposure and allows us to apply stricter hardening (API keys, IP allowlists, rate limits) without impacting the core API.
- Independent lifecycle: Payment features can be deployed, scaled, and audited independently of core releases, with targeted observability and on-call runbooks for payment incidents.

7.5.2. Service Architecture

Tech stack: NestJS 11 (TypeScript 5.7), PostgreSQL 17, Redis 7, Prisma ORM, Winston logging, AES-256-GCM encryption.

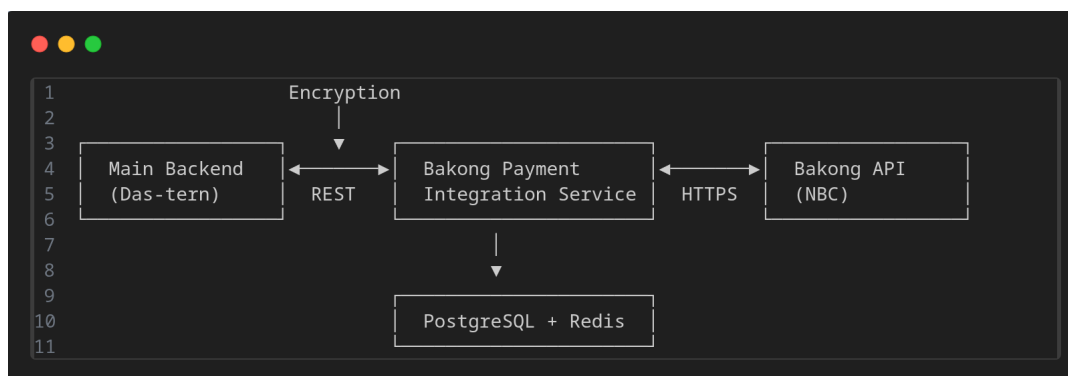


Figure 26: Service Architecture

❖ **Responsibilities (Bakong Service):**

- **KHQR generation** (EMV-compliant) and deep link building
- **Payment status** monitoring (polling with backoff)
- **Subscription lifecycle** (create, renew, upgrade/downgrade, cancel)
- **Proration** for plan changes
- **Audit** of all payment operations
- **Security** controls: API key auth, rate limiting, IP blocking, data encryption

7.5.3. Integration Boundary with the Main Backend

Only the **Main Backend** talks to the **Bakong Service** (never directly to the Bakong API). Communication uses **REST + API key** over TLS. The Main Backend:

1. **Creates** a payment for a user/plan.
2. **Displays** the returned KHQR/deep link to the mobile app.
3. **Polls** or receives webhook-style updates (via a secure callback or explicit polling) from the Bakong Service.
4. **Activates/Renews** subscriptions when status becomes **PAID**.

7.5.4. Subscription Management APIs

- **Get Subscription Status:** GET /api/subscriptions/status/:userId
- **Upgrade (prorated):** POST /api/subscriptions/upgrade
 - Returns requiresPayment=true and a new KHQR if additional amount is due.
- **Downgrade:** POST /api/subscriptions/downgrade (schedules change at next billing)
- **Cancel:** POST /api/subscriptions/cancel
- **Renew:** POST /api/subscriptions/renew (manual trigger after payment confirmation)

7.5.5. Payment Lifecycle & State Machine

States: PENDING → (PAID | FAILED | TIMEOUT | EXPIRED)

❖ **Flow (Happy Path):**

1. Backend requests **Create Payment**.
2. Bakong Service generates **KHQR + billNumber + md5Hash**; persists PENDING.
3. User scans KHQR with Bakong app and pays.

4. Bakong Service detects payment (polling/webhook), **verifies** metadata/signature, updates DB to **PAID**.
5. Service **notifies** the Main Backend (callback or the backend polls /status/:md5).
6. Main Backend **activates/renews** subscription and records an **AuditLog** event.

Timeout/Expiry: If no payment within the configured SLA, status moves to **TIMEOUT** or **EXPIRED** and the QR is no longer valid.

7.5.6. KHQR Generation & Deep Links

- KHQR strings are generated to **EMV-coherent** format for Bakong scanning.
- The service also returns a **deep link** (bakong://...) so mobile clients can open Bakong directly when supported.
- The raw KHQR and an image (png) path are returned for in-app rendering. [\[35\]](#)

7.5.7. Security Design

1. API Key Authentication

- a. All non-health endpoints require Bearer API key.
- b. Keys are **rotatable** and **scoped** (production vs. staging).
- c. Failed attempts tracked in Redis; **IP block** after threshold (e.g., 10 failures/5 minutes).

2. Rate Limiting

- a. 100 req/min per API key/IP (Redis distributed counters).
- b. **429** with Retry-After when exceeded.

3. Data Encryption

- a. Sensitive fields encrypted with **AES-256-GCM** at rest.
- b. Keys derived via **PBKDF2** ($\geq 100k$ iterations); additional **HMAC-SHA256** integrity tags.
- c. One-way hashing (MD5 bill identifier used **only** as an external lookup key; payment secrets never exposed).

4. Input Validation & Hardening

- a. DTO validation (required fields, types, array limits).
- b. SQL injection protection via Prisma parameterization.
- c. Strict CORS and **IP allowlist** on the service firewall (only Main Backend IPs).

5. Audit Trail

- a. Every payment/ subscription mutation is logged with user, action, resource, payload digest (no secrets), timestamp, and outcome.
- b. **Security log** channel for auth failures and rate-limit blocks.

7.5.8. Resilience, Idempotency & Error Handling

- **Idempotency keys** (e.g., Idempotency-Key header mirroring billNumber) ensure **Create Payment** is safe to retry.
- **Exponential backoff** and **jitter** on Bakong API polling; **circuit breaker** trips on repeated upstream failures.
- **Outbox pattern** for guaranteed delivery of “payment confirmed” notifications to the Main Backend (retries until ACK).
- **Clock skew tolerance** in signature verification and timeout windows.
- **Graceful degradation**: If Bakong is degraded, the service responds with a clear status and advice for the client to retry later.

7.5.9. Health, Observability & Runbooks

- **/api/health**, **/api/health/ready**, **/api/health/live** expose liveness/readiness.
- Structured JSON logs (Winston) to combined.log, error.log, and security.log.
- Dashboards track: request rate, success/error %, median/p95 latency, Redis memory, DB pool usage, and queue depth.
- **Runbook** includes: rotating API keys, unblocking IPs, replaying failed callbacks, reconciling payments (bulk status check).

7.6. Medication Reminder Generation Logic

Using the structured medication data and user mealtimes, DasTern automatically generates reminder schedules.

- Example Reminder Calculation:

Medicine	Frequency	Timing	Generated Reminder Time
Paracetamol	3/day	After meal	08:15, 13:15, 19:45
Amoxicillin	2/day	Before meal	07:45, 19:15

Table 21: Reminder Calculation Example

❖ **The reminder logic accounts for:**

- Meal-based offsets (before/after meal)
- Multiple daily doses
- Consistent spacing between doses

7.6.1. Notification and Monitoring Integration

❖ **Once reminders are generated:**

- push notifications are scheduled locally (offline supported)
- Missed doses trigger alert logic

7.7. UX/UI (Low-Fidelity and High-Fidelity)

The user interface and user experience of the DasTern application were developed through an iterative prototyping process.

- **Low-fidelity wireframes** were first created to define basic layouts, navigation flow, and screen hierarchy for patients, doctors, and family members.
- **High-fidelity Kiri-OCRs** were then designed using Figma to represent the final look and feel of the application, including colors, icons, typography, and interaction behaviors.

Special attention was given to usability and accessibility, particularly for elderly users and individuals with limited digital literacy. Khmer language support, large buttons, simple navigation, and visual clarity were emphasized throughout the design. As a result, approximately **95% of the full application Kiri-OCR** has been completed in Figma, covering all major user flows.

7.7.1. Patient Interface Implementation

The Patient Interface was developed using Flutter to provide cross-platform mobile support for Android and iOS devices. The implementation follows a modular architecture separating UI components, business logic, and data services to ensure scalability and maintainability. [\[34\]](#)

- **Home Dashboard**

The Home Dashboard serves as the central overview screen of the application. It provides:

- Today's medication status summary
- Upcoming reminders
- Missed dose alerts
- Quick access to scan or view schedules

Color indicators (e.g., green for taken, red for missed) were used to visually represent medication adherence status, improving usability and reducing cognitive load.

- **OCR Scan Screen**

The OCR Scan Screen allows users to capture prescription images using the device's camera. The implementation includes:

- Real-time camera integration
- OCR text extraction processing
- Processing feedback indicator (loading state)
- Editable extracted medication list

After scanning, users can review and edit extracted data before confirming. This validation step ensures data accuracy and reduces medication entry errors.

- **Medication Schedule Screen**

This screen presents a structured daily medication timeline. Features include:

- Time-based medication grouping
- Taken / Missed indicators
- Tap-to-confirm medication intake
- Auto-status update after reminder confirmation

The timeline format helps users clearly understand when to take each medication throughout the day.

- **Reminder Confirmation Screen**

This screen displays a preview of reminders based on user-defined meal preferences (e.g., before breakfast, after lunch). It allows:

- Confirmation of reminder timing
- Visual preview of notification schedule
- Adjustment before final submission

This ensures reminders are correctly aligned with patient routines.

- **Settings Screen**

The Settings Screen allows personalization of user preferences, including:

- Mealtime configuration (custom breakfast, lunch, dinner times)
- Language selection
- Notification enable/disable controls
- Account and subscription management

The system dynamically adjusted reminder schedules based on these variations, confirming the flexibility and correctness of the reminder algorithm.

7.7.2. Doctor Interface Implementation

The DasTern system implements a comprehensive doctor interface to manage, keep track patient, create prescriptions for patients built with **Flutter** [\[34\]](#) for the mobile frontend and **Nest.js** backend for API services. The architecture follows a client-server model using RESTful APIs, PostgreSQL database, Prisma ORM for database access, and using OAuth/Google Sign-In for Authentication. These are following features which have been implemented in.

- **System Architecture**

System has three main providers handling the states with “DoctorDashboardProvider” managing all dashboards, “PrescriptionProvider” handles prescriptions creation, and “ConnectionProvider” manages doctor-patient connections.

```

lib/
├─ ui/screens/doctor/      → All doctor screens and tabs
├─ ui/widgets/             → Reusable UI components
├─ models/                 → Data model classes
├─ providers/              → State management
├─ services/               → API, database, and sync services
└─ l10n/                  → Khmer and English localization files

```

Figure 27: Flutter project folder structure

```

src/modules/
├─ doctor-dashboard/       → Controller, service, DTOs for dashboard
├─ connections/           → Controller, service, DTOs for connections
└─ prescriptions/         → Controller, service, DTOs for prescriptions

```

Figure 28: NestJS backend folder structure

● Doctor Authentication

The authentication system implements secure doctor login with multi-factor validation including full name verification, phone number authentication, and password encryption.

- Login interface with full name, phone number and password authentication
- Form validation for secure access
- Form Validation Features
- Real-time validation: Checks for empty fields before submission
- Khmer language support: UI labels displayed in Khmer (ឈ្មោះ for first name, ឈ្មោះ for last name)
- Secure input: Password fields are masked for security
- Loading states: Provides visual feedback during authentication process
- Navigation to doctor dashboard upon successful login

● Doctor Dashboard and Analytics

The doctor dashboard interface is implemented in “doctor_home_tab.dart” and managed by “DoctorDashboardProvider”. The dashboard displays the summary of current patient activity such as; total of connected patients, number of patients has low adherence. It also shows the graph representing the trend of patients who have low and high adherence. Those trends can be shown daily and monthly.

- API Endpoints:

Method	Endpoint	Description
GET	/doctor/dashboard	Return total patients, alters, pending requests, recent activity
GET	/doctor/dashboard/graph?period=week month	Return adherence graph data points

Table 22: Doctor Dashboard API Endpoints

The dashboard overview response contains: “totalPatients”, “patientNeedingAttention”, “todayAlerts”, “recentActivity”, and “pendingRequests”. The graph endpoint returns an array of data points each containing a date, adherence percentage, and number of patients tracked.

The dashboard provides a comprehensive overview of patient medication adherence with real-time statistics, alert notifications, and visual analytics.

- Localized header with doctor identification and notifications
- Statistics cards showing patients who received medication (20) vs. pending (04)
- Alert notifications listing non-adherent patients with missed dose details
- Interactive bar chart displaying weekly adherence data with Khmer day labels
- Color-coded visualization (Blue: received, Red: missed)

- **Patient Records and History**

Patient management is divided into two screens; “DoctorPatientTab (doctor_patient_tab.dart)” presents a searchable, filterable list of all connected patients, and “PatientDetailScreen (patient_detail_screen.dart)” providing the details of a single person.

- Patient List

The patient list supports search by name and filtering by adherence level using filter chips labeled All, Good, Moderate, and Poor. The list is built with “ListView.builder” and supports pull-to-refresh. Each patient card shows their name, age, number of active prescriptions, and their current adherence level with a color indicator. Adherence levels are classified as follows:

Level	Threshold	Color
Good	Above 80%	Green
Moderate	60% to 80%	Yellow
Poor	Below 60%	Red

Table 23: Patients Threshold

The patient list is fetched from GET /doctor/patients with support for pagination, search, and sort query parameters. The response is paginated, and each item contains the patient's full profile, active prescription count, adherence percentage, and last activity timestamp.

- Patient Detail View

Tapping a patient opens the “PatientDetailScreen”, which is organized into four tabs: Overview, Prescriptions, Notes, and Activity. The Overview tab displays the patient's demographics and current adherence score. The Prescriptions tab lists all active and past prescriptions for that patient. The Notes tab shows all doctor notes written for that patient. The Activity tab shows a timeline of recent adherence events.

All this data is fetched in a single call to GET /doctor/patients/:patientId/details, which returns a composite response containing the patient's profile, adherence result, prescriptions, vitals, alerts, and notes. Adherence can also be queried separately for a custom date range using GET /doctor/patients/:patientId/adherence.

- Patient Detail API Endpoints

Method	Endpoints	Description
GET	/doctor/patients/:patientId/details	Full patient profile, vitals, prescriptions, notes
GET	/doctor/patients/:patientId/adherence	Adherence data with custom date range filter

Table 24: Patients Detail API Endpoints

- Prescription Creation Interface

Prescription management is handled across three screens: “CreatePrescriptionScreen” for creating new prescriptions, “DoctorPrescriptionHistoryTab” for browsing all prescriptions, and PrescriptionViewScreen for viewing full prescription details. All prescription state is managed by PrescriptionProvider.

- Creating a Prescription

The prescription form in “CreatePrescriptionScreen” is divided into five sections. First, the doctor selects a patient from a dropdown of their connected patients, with the patient's age calculated and displayed automatically from their date of birth. Second, the doctor fills in clinical information including chief symptoms (required), diagnosis (required), and clinical notes (optional). Third, the doctor adds one or more medications using a dynamic list where medications can be added or removed. Each medication entry captures the medicine name in both English and Khmer,

medicine type, unit, dosage amount, timing schedule across morning, afternoon, evening, and night, before or after meal flags, a PRN (as needed) flag, and duration in days. Fourth, an optional follow-up date can be set. Fifth, the form is validated and submitted, with loading state and success or error feedback displayed to the user.

Form validation is enforced both on the Flutter side and at the NestJS backend using “CreatePrescriptionDto” with class-validator decorators. The backend also verifies that the doctor has an accepted connection with the selected patient before allowing the prescription to be created.

- **Prescription History and Viewing**

The “DoctorPrescriptionHistoryTab” displays all prescriptions created by the doctor. The list can be filtered by status (DRAFT, ACTIVE, COMPLETED, ARCHIVED) and by a specific patient. Sorting by date, patient name, or status is also supported. Tapping a prescription opens the “PrescriptionViewScreen”, which shows the full prescription header, a detailed medications table, the doctor's license number, the patient's acknowledgment status, and edit or update buttons.

The system also supports urgent prescription updates, where the doctor can flag a change as urgent. Every update to a prescription creates a version of snapshot stored in the “prescription_versions” table, maintaining a full audit trail of prescription changes.

- **Prescription API Endpoints**

Method	Endpoints	Description
POST	/prescriptions	Create a new prescription
GET	/prescriptions	Retrieve all prescriptions with optional status and patient filters
GET	/prescriptions/:id	Retrieve a single prescription by ID
PATCH	/prescriptions/:id	Update an existing prescription
POST	/prescriptions/:id/urgent-update	Flag an update as urgent
DELETE	/prescriptions/:id	Delete a prescription

Table 25: Prescription API Endpoints

- **Doctor-Patient Connection Management**

Before a doctor can access any patient data or create prescriptions for a patient, an accepted connection must exist between them. This is enforced at the backend level on every protected endpoint. The connection lifecycle supports four statuses: PENDING, ACCEPTED, and REVOKED.

Pending connection requests sent by patients are visible on the dashboard and in a dedicated pending list screen. The doctor can accept or reject each request. When accepting, the doctor can also set a permission level for that patient: NOT_ALLOWED, REQUEST, SELECTED, or ALLOWED. Connections can be revoked at any time, which immediately removes the doctor's access to that patient's data.

- **Connection API Endpoints**

Method	Endpoint	Description
GET	/doctor/connections/pending	List all pending connection requests
POST	/doctor/connections/:id/accept	Accepts a connection and set permission level
POST	/doctor/connections/:id/reject	Reject a connection with optional reason
POST	/doctor/connections/:id/disconnect	Revoke an existing connection

Table 26: Connection API Endpoints

Prescription history: Chronological list of all prescriptions

- Medication timeline: Visual timeline of medication administration
- Adherence trends: Graphical representation of compliance over time

This design aligns with advisor feedback emphasizing **what doctors want to know quickly**.

7.7.3. Family Interface Implementation

The Family Interface is a comprehensive feature that empowers caregivers and family members to monitor patient medication adherence and health status while ensuring privacy and security. The system allows patients to grant controlled access to their medication data to designated caregivers through a secure connection process, with granular permission levels that dictate what actions caregivers can perform.

- **Family Dashboard**

- Linked patient summary: Display a list of a patient basic information info (name, age, phone number of patient)
- Daily adherence status: Visual summary of the patient's medication intake for the current day (taken, missed, upcoming).
- Patient Timeline View: Chronological log of medication events, including timestamps for taken and missed doses.
- Notification Log & Alerts: Family members receive alerts through the app's notification system. These notifications include medication alerts and reminders.

- **Patient Linking Process**

The family member must link to patient account using one of the following methods:

- Family members can link to a patient by scanning a QR code generated by the patient or entering a unique patient ID. This initiates a secure connection request, which the patient must approve.
- Patients receive and approve or reject connection requests via their dashboard, ensuring control over who can view their data.

➤ **Family Connection API Endpoints**

Method	Endpoint	Purpose
POST	/api/v1/connections/tokens/generate	Patient generates a share token with selected permission level
POST	/api/v1/connections/tokens/validate	Validate token before showing connection preview
POST	/api/v1/connections/tokens/consume	Convert valid token into a pending connection request
POST	/api/v1/connections	Create direct connection request (non-token flow)
PATCH	/api/v1/connections/:id/accept	Patient approves request and sets permission level
PATCH	/api/v1/connections/:id/revoke	Revoke connection and immediately remove access
PATCH	/api/v1/connections/:id/permission	Update caregiver permission after connection is active
GET	/api/v1/connections/patients	List patients linked to the current caregiver
GET	/api/v1/connections/caregivers	List caregivers linked to the current patient
GET	/api/v1/connections/caregiver-limit	Return current and maximum allowed family links by plan
GET	/api/v1/connections/history	Return connection history/timeline records

Table 27: Family Connection API Endpoints

➤ **Family Dashboard Data API Endpoint**

Method	Endpoint	Purpose
GET	/api/v1/doses/caregiver/:patientId	Return patient dose schedule/status for caregiver dashboard
GET	/api/v1/adherence/today	Return daily adherence summary
GET	/api/v1/adherence/weekly	Return weekly adherence summary
GET	/api/v1/adherence/monthly	Return monthly adherence summary

Table 28: Family Dashboard Data API Endpoint

➤ **Notification and Reminder**

Method	Endpoint	Purpose
POST	/api/v1/connections/nudge	Send reminder nudge from caregiver to patient
POST	/api/v1/connections/nudge/respond	Send patient response back to caregiver
PATCH	/api/v1/connections/:id/alerts	Enable or disable alerts for a specific connection
GET	/api/v1/notifications	Retrieve notifications and unread count
PATCH	/api/v1/notifications/:id/read	Mark notification as read
DELETE	/api/v1/notifications/:id	Delete notification entry

Table 29: Notification and Reminder

➤ **Implementation Summary**

Aspect	Implementation
Core Objective	Enable family/caregiver monitoring while keeping patient control and privacy
System Layers	Frontend (Flutter UI), Backend (NestJS APIs), Database (PostgreSQL with Prisma).
Permission Levels	View only, View & Remind, View & Manage
Linking Methods	QR/token-based linking plus manual token entry
Security	JWT-protected endpoints with server-side connection and permission validation
Data Persistence	Connection, token, dose event, and notification records stored with status and timestamps.
Dashboard Data	Caregiver sees linked patient info, daily dose states, and timeline-based medication events
Alerts and Reminders	In app notifications for connection events, missed doses, and caregiver nudges

Table 30: Implementation Summary

7.8. Setting Implementation

The Settings page covers three main features: Edit Profile, Change Password, and Subscription & Billing. Each feature is connected to specific environment settings that control how it works. The remaining settings features such as appearance, notifications, and sign out are simpler and summarized at the end.

Section	Priority	Key Settings
Edit Profile	High	DATABASE_URL, ENCRYPTION_KEY, SENDGRID_API_KEY
Change Password	High	JWT_SECRET, DATABASE_URL, LOG_LEVEL
Subscription & Billing	High	STRIPE_SECRET_KEY, BAKONG_API_KEY, PREMIUM_PRICE_USD
Appearance (Theme/Lang)	Low	DEFAULT_LANGUAGE, DATABASE_URL
Notification Settings	Medium	FCM_SERVER_KEY
Sign Out / Delete	Low	JWT_SECRET, DATABASE_URL

Table 18: Show feature and its priority

7.8.1. Edit Profile feature

Users can update their first name, last name, phone number, and date of birth. Email changes trigger a verification email via SendGrid. All changes are saved to the database and logged in the audit trail.

+ API Endpoint

```
PATCH /api/v1/users/profile
Body: { firstName, lastName, phone, dateOfBirth }
Response: { user: User, message: 'Profile updated' }
```

This is the API call the Flutter app makes when the user taps Save. PATCH means it only updates the fields provided it does not replace the entire user record. The app sends the changed fields in the Body, and the server responds with the updated user object.

+ How It Works

The process is split between the Flutter app and the NestJS backend

Frontend	The screen loads the current user data into text fields. When the user taps Save, the app sends a PATCH request with the updated fields and a JWT token in the Authorization header.
Backend	The screen loads the current user data into text fields. When the user taps Save, the app sends a PATCH request with the updated fields and a JWT token in the Authorization header.

Table 19: Summary Table of how it work

7.8.2. Change Password

Users enter their current password and a new password to update their account credentials. After a successful change the user stays logged in.

+ API Endpoint

```
POST /api/v1/auth/change-password
Body: { currentPassword, newPassword }
Response: { message: 'Password changed successfully' }
```

POST is used here because the server is performing an action, not just reading or updating a record. The user is identified from the JWT token so only the two passwords need to be sent in the body.

+ How It Works

Four steps run in sequence. If any step fails the process stops and returns an error.

Validate	The app checks locally that the new password matches the confirm field and is at least 8 characters before sending anything to the server.
Verify	The backend fetches the user's stored passwordHash and uses bcrypt.compare() to confirm the current password is correct.
Save	The backend runs bcrypt.hash(newPassword, 10) and saves the new hash to the users table.
Log	A PASSWORD_CHANGED entry is written to the audit log with the user ID and timestamp.

Table 20: Implementation Flow

7.8.3. Subscription & Billing

Users can view their current plan and upgrade to Premium (\$0.50/month) or Platinum (\$1.00/month). Cambodian users pay via Bakong QR code. After payment is confirmed by the provider, the backend automatically upgrades the user's tier in the database.

Feature	Freemium	Premium	Platinum
Alerts	Yes	Yes	Yes
Family & Doctor connection	No	Yes	Yes
Price	\$0	\$0.50 / month	\$1.00 / month
Group plan	No	No	Yes
OCR Scan	No	Yes	Yes
Get Report	No	Yes	Yes

Table 21: Subscription Plan table

+ API Endpoints

The GET endpoint loads the current plan when the screen opens. The two POST endpoints start each payment flow. The two webhook endpoints are called automatically by Bakong and Stripe after a payment completes.

```
GET /api/v1/subscriptions/current      # Load current plan and storage
POST /api/v1/subscriptions/bakong/create # Generate Bakong QR code
POST /api/v1/subscriptions/stripe/create # Create Stripe checkout session
POST /api/v1/webhooks/bakong           # Bakong confirms payment
POST /api/v1/webhooks/stripe           # Stripe confirms payment
```

+ How It Works

Bakong Payment : Backend calls Bakong API with BAKONG_API_KEY to get a QR code. User scans it with their bank app. Bakong sends a webhook the backend verifies the signature using BAKONG_WEBHOOK_SECRET then upgrades the user's tier.

Stripe Payment : Backend creates a Stripe checkout session using STRIPE_SECRET_KEY. User enters card details. Stripe sends a webhook the backend verifies it with STRIPE_WEBHOOK_SECRET then upgrades the tier.

7.8.4. Other Settings Features

The remaining settings features are simpler. The table below summarizes each one in a single row.

Feature	How It Works	Key Setting
Appearance / Language	User choice saved to device storage and synced to DB via PATCH /users/preferences.	DEFAULT_LANGUAGE
Notification Settings	Toggle switches save preferences to DB. FCM delivers push notifications at the scheduled time.	FCM_SERVER_KEY
Two-Factor Auth	Backend generates a TOTP secret and QR code. User scans it with an authenticator app and confirms with a 6-digit code.	JWT_SECRET
Sign Out	Clears JWT from device storage and blacklists it in Redis so it cannot be reused.	REDIS_HOST
Delete Account	Calls DELETE /users/account after confirmation. Permanently removes all user data from the database.	DATABASE_URL

Table 22: Summarizes simpler features

VIII. CONCLUSION

DasTern has officially transitioned from a design concept into a fully functioning digital health platform. To solve post-discharge medication mismanagement, we built a system that scans traditional paper prescriptions and automatically converts them into clear digital schedules. By combining a local OCR pipeline with an AI text processing model (**Ollama running Llama 3.2 3B**), DasTern successfully corrects recognition errors and formats prescriptions accurately in Khmer.

Throughout the project, our team successfully integrated all major technical components. The Flutter mobile app links directly to a live Nest.js backend and a secure PostgreSQL database. We fully deployed the Python OCR microservice, AI extraction engine, Doctor Dashboard and Bakong payment service. Together, these components create a complete, working ecosystem that connects patients, doctors, and caregivers.

Following rigorous system integration and usability testing, we translated our Figma designs into a highly accessible application. Core features including offline data synchronization, Khmer text-to-speech reminders, and simplified navigation are fully operational. This ensures that elderly users and those in rural areas can easily manage their medication without requiring an internet connection.

In conclusion, DasTern fulfills the technical and practical requirements for Capstone II. We have delivered a secure, scalable health technology solution that makes the transition from hospital care to home recovery safer, ultimately reducing medication errors and improving healthcare outcomes across Cambodia.

REFERENCES

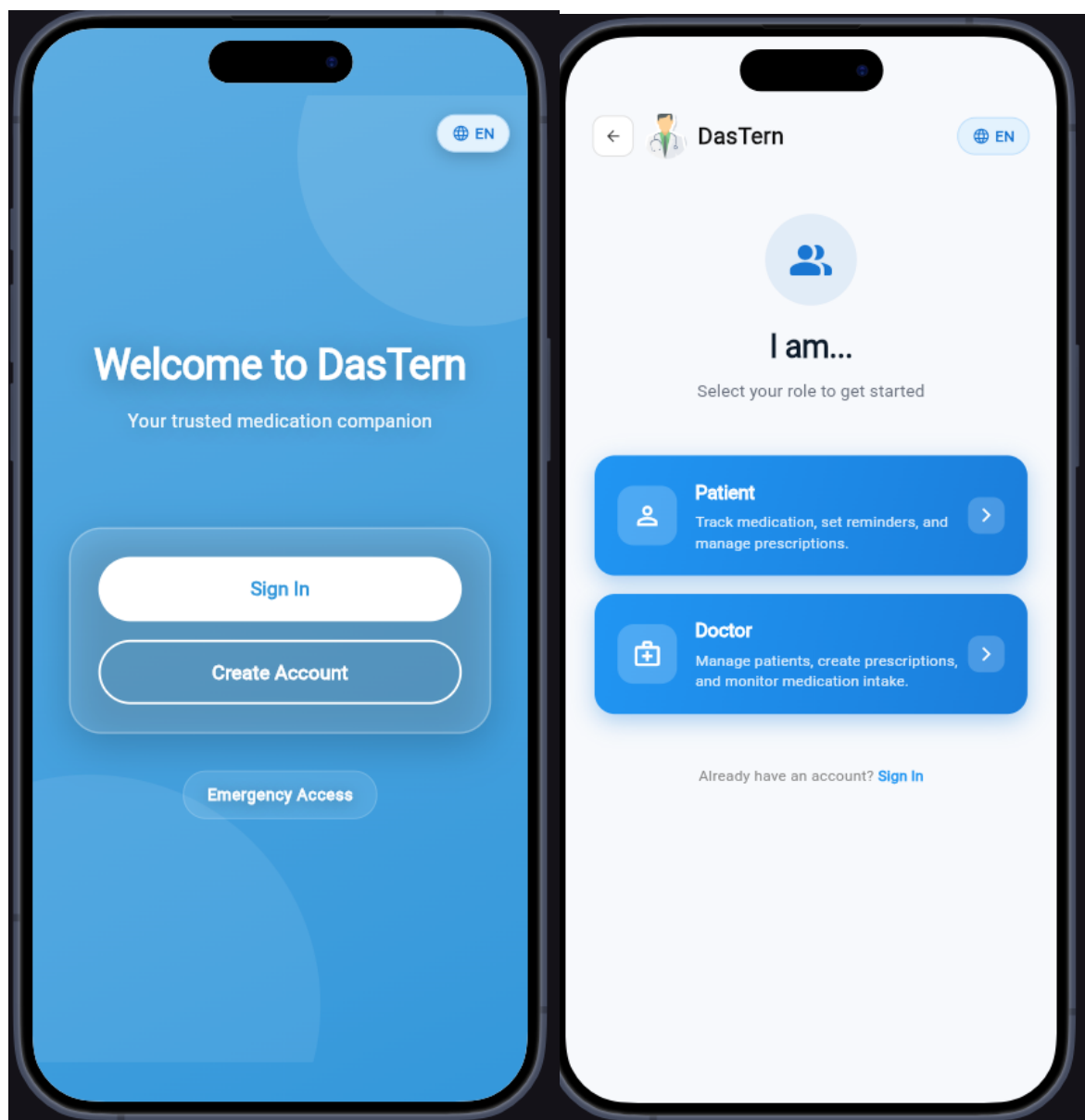
- [1] Thakkar, J., Kurup, R., Laba, T. L., Santo, K., Thiagalingam, A., Rodgers, A., Woodward, M., Redfern, J., & Chow, C. K. (2016). *Mobile telephone text messaging for medication adherence in chronic disease: A meta-analysis*. **JAMA Internal Medicine**, 176(3), 340–349. [Link](#)
- [2] Sabaté, E. (2003). *Adherence to long-term therapies: Policy for action*. WHO.
- [3] World Health Organization. (2003). *Adherence to long-term therapies: Evidence for action*. WHO Press. [Link](#)
- [4] Grindrod, K. A., Li, M., & Gates, A. (2018). Medication adherence apps: A quality assessment. *JMIR mHealth and uHealth*, 2(1), e32. [Link](#)
- [5] Sokol, M. C., McGuigan, K. A., Verbrugge, R. R., & Epstein, R. S. (2005). *Impact of medication adherence on hospitalization risk and healthcare cost*. **Medical Care**, 43(6), 521–530. [Link](#)
- [6] Free, C., Phillips, G., Watson, L., Galli, L., Felix, L., Edwards, P., Patel, V., & Haines, A. (2013). *The effectiveness of mobile-health technologies to improve health care service delivery processes: a systematic review and meta-analysis*. *PLoS Medicine*, 10(1), e1001363. [Link](#)
- [7] Vervloet, M., Linn, A. J., van Weert, J. C. M., de Bakker, D. H., Bouvy, M. L., & van Dijk, L. (2012). *The effectiveness of interventions using electronic reminders to improve adherence to chronic medication: A systematic review of the literature*. *BMJ*, 344, d8219. [Link](#)
- [8] Park, J. Y. E., Li, J., Howren, A., Tsao, N. W., & De Vera, M. (2019). *Mobile phone apps targeting medication adherence: quality assessment and content analysis of user reviews*. *JMIR mHealth and uHealth*, 7(1), e11919. [Link](#)
- [9] Patel, R., Shah, M., & Desai, S. (2019). *OCR-based medical prescription recognition using image preprocessing techniques*. *International Journal of Computer Applications*.
- [10] Singh, A., & Kaur, P. (2020). Text recognition in medical documents using OCR and NLP techniques. *Journal of Healthcare Informatics Research*, 4, 275–290. [Link](#)
- [11] DiMatteo, M. R. (2004). Social support and patient adherence to medical treatment: A meta-analysis. *Medical Care*, 42(3), 200–209. [Link](#)
- [12] Choudhry, N. K., Isaac, T., & Lauffenburger, J. C. (2017). Understanding the impact of family involvement on medication adherence. *Circulation*, 136(13), 1247–1250. [Link](#)

- [13] Piette, J. D., Striplin, D., Marinec, N., Chen, J., Trivedi, R. B., Aron, D. C., Fisher, L., & Aikens, J. E. (2015). *A mobile health intervention supporting heart failure patients and their informal caregivers: A randomized comparative effectiveness trial. Journal of Medical Internet Research*, 17(6), e142. [Link](#)
- [14] Healthera. (n.d.). *Healthera digital healthcare platform*. Retrieved from [Link](#)
- [15] NowPatient. (n.d.). *NowPatient digital health service*. Retrieved from [Link](#)
- [16] Google Cloud, "Document AI Documentation," Google Cloud, [Online]. Available: [Link](#)
- [17] Google, "Flutter Documentation," Google Developers, [Online]. Available: [Link](#)
- [18] Google, "Nest.js Documentation," Website, [Online]. Available: [Link](#)
- [19] OpenJS Foundation, "Node.js Documentation," Node.js, [Online]. Available: [Link](#)
- [20] Telegram, "Telegram Bot API," Telegram Developers, [Online]. Available: [Link](#)
- [21] Buoy, Rina, Sokchea Kor, and Nguonly Taing. "An end-to-end Khmer optical character recognition using sequence-to-sequence with attention." arXiv preprint arXiv:2106.10875 (2021). [Link](#)
- [22] mrrtmob, "Kiri OCR," Hugging Face, 2026. [Online]. Available: <https://huggingface.co/mrrtmob/kiri-ocr>. Available: [Link](#)
- [23] Nonogaki, A., Heang, H., Yi, S., van Pelt, M., Yamashina, H., Taniguchi, C., Nishida, T., & Sakakibara, H. (2019). *Factors associated with medication adherence among people with diabetes mellitus in poor urban areas of Cambodia: A cross-sectional study. PLoS ONE*, 14(11), e0225000.
- [24] Yi, S., et al. (2019). *Patients' and healthcare providers' perspectives of diabetes management in Cambodia: A qualitative study. BMC Health Services Research*, 19, 448.
- [25] Savina, C., Buffel, V., Van Olmen, J., et al. (2023). *Understanding barriers to healthcare access for older adults in Cambodia. International Journal of Integrated Care*, 23.
- [26] UNICEF. (2023). *Strengthening community health in Cambodia through digitalization*. United Nations Children's Fund.
- [27] Te Velde, D. W., et al. (2025). *Self-care and health-seeking for diabetes and hypertension in Cambodia. Health Policy and Planning*, Oxford Academic.
- [28] Nonogaki, A., Heang, H., Yi, S., van Pelt, M., Yamashina, H., Taniguchi, C., Nishida, T., & Sakakibara, H. (2019). *Factors associated with medication adherence among people with diabetes mellitus in poor urban areas of Cambodia: e0225000. Link*
- [29] World Health Organization. (2025). *1 in 5 Cambodian adults are at risk of stroke as noncommunicable disease risks increase. Western Pacific Region. Link*

- [30] Chham, S., Buffel, V., Van Olmen, J., Chhim, S., Ir, P., & Wouters, E. (2022). *The cascade of hypertension care in Cambodia: Evidence from a cross-sectional population-based survey*. **BMC Health Services Research**, 22, 838. [Link](#)
- [31] Choeng Rayu, “DasTern OCR service anti module: System architecture and implementation,” GitHub repository, 2025. [Online]. [Link](#)
- [32] Choeng-Ray, “DasTern AI-LLM service module: System architecture and implementation,” GitHub repository, 2025. [Online]. Available: [Link](#)
- [33] Choeng-Ray, “DasTern backend system implementation (Nest.js),” GitHub repository, 2025. [Online]. Available: [Link](#)
- [34] Choeng-Ray, “DasTern mobile front end (Flutter) implementation,” GitHub repository, 2025. [Online]. Available: [Link](#)
- [35] Choeng-Ray, “DasTern payment method module (Bakong integration),” GitHub repository, 2025. [Online]. Available: [Link](#)

APPENDICES

Login Interface:





 **DasTern**

 EN

Create New Account

Step 2 of 2 - Account Information

Email

Phone Number (Optional)

 +855

Password

Confirm Password

Please read the terms and conditions before using the app

☒ Already read

Create Account

or register with

 Register with Google

 Register with Telegram

Already have an account? [Sign In](#)



 **DasTern**

 EN

Create New Account

Step 1 of 2 - Personal Information

Last Name

First Name

Gender

Male

Female

Other

Date of Birth

ID Card Number (Optional)

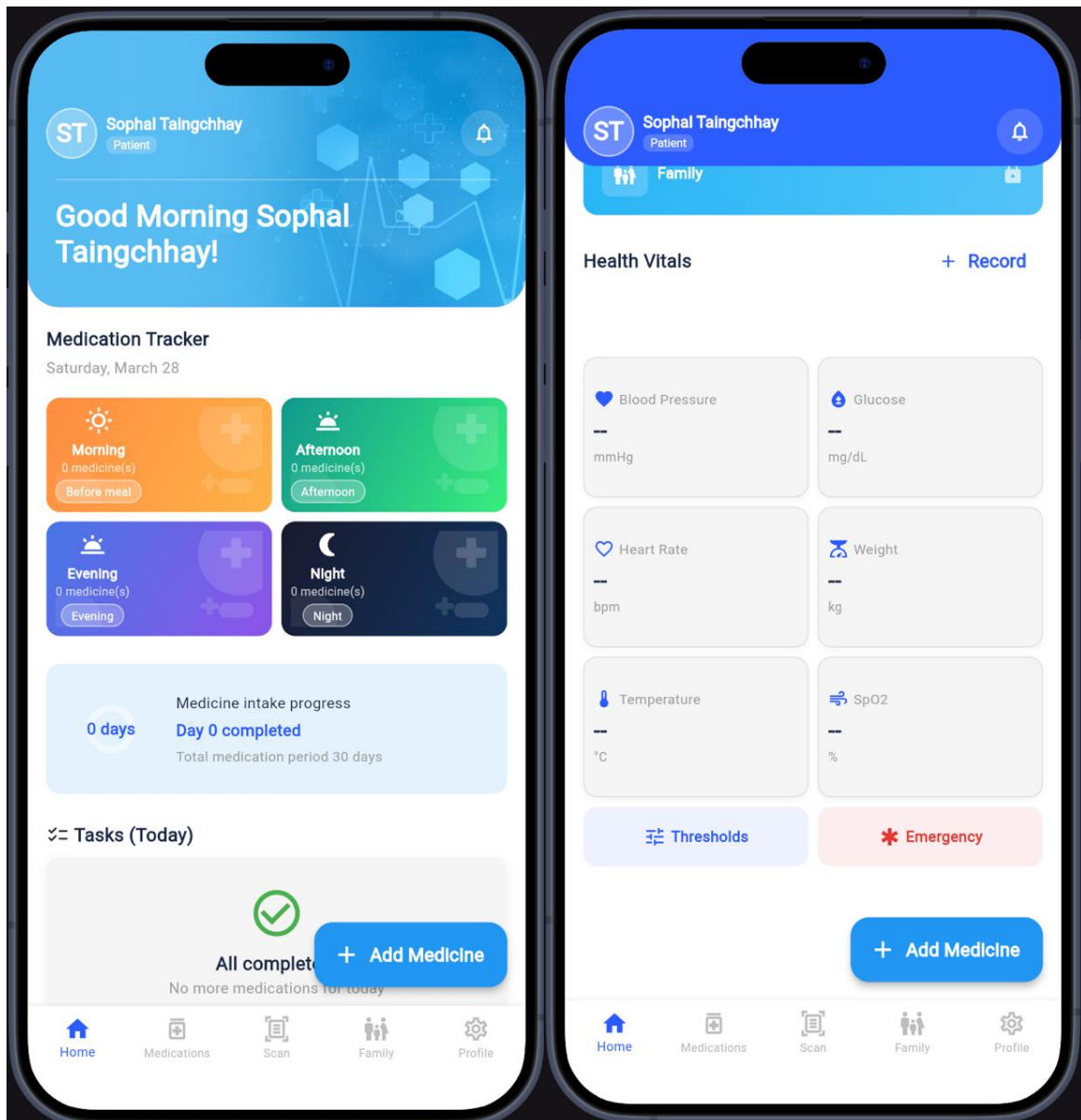
Continue

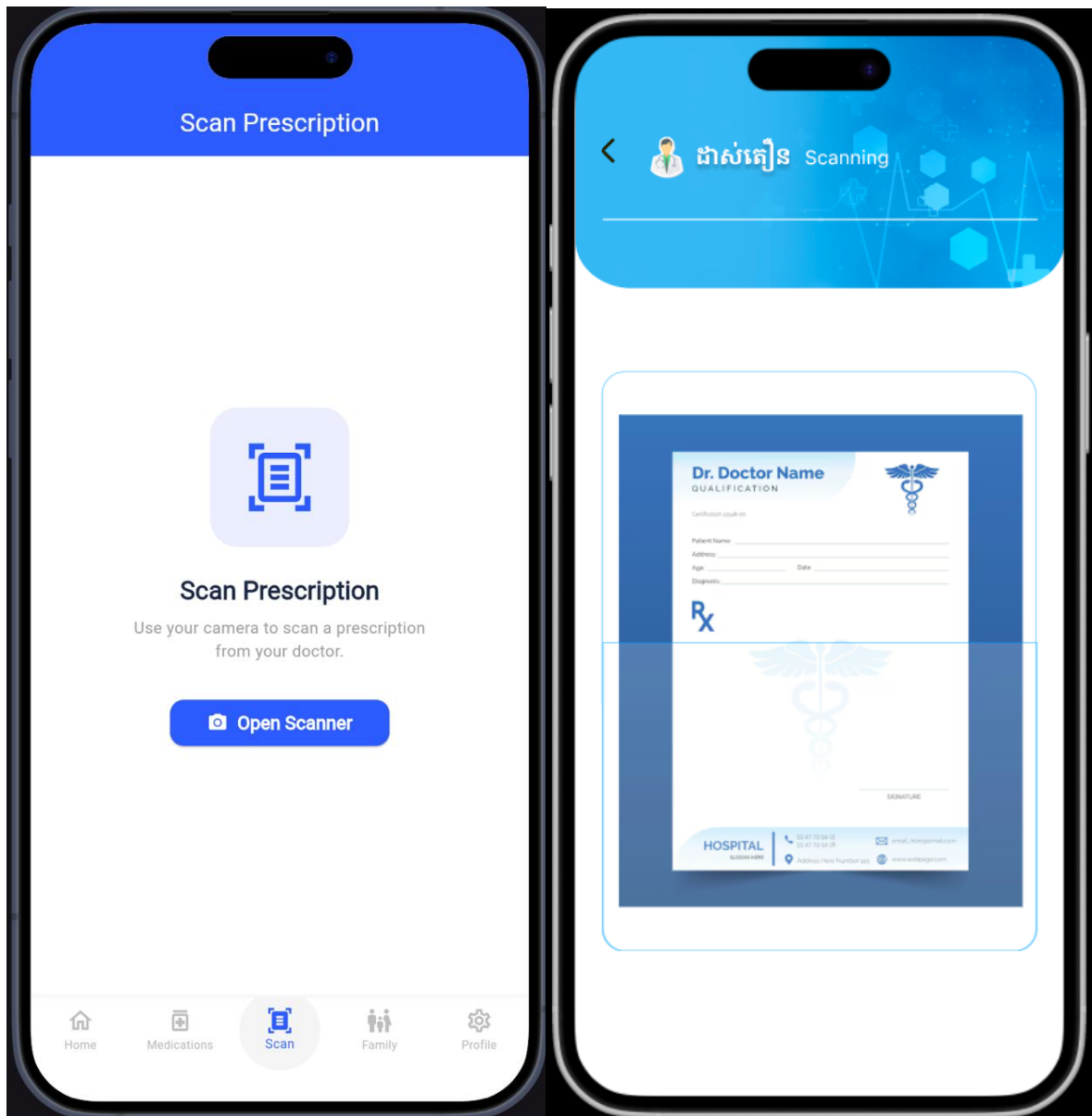
or register with

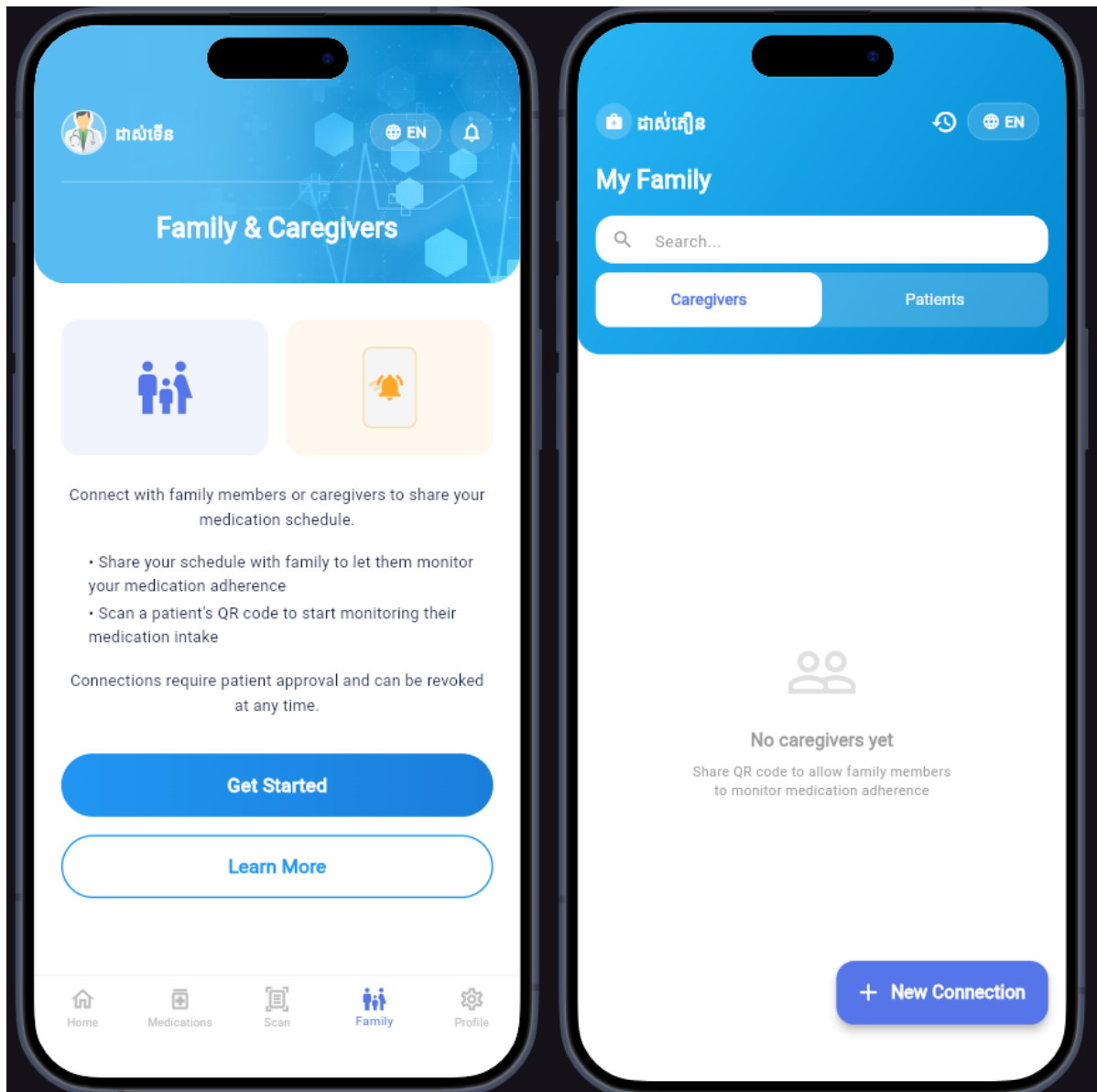
 Register with Google

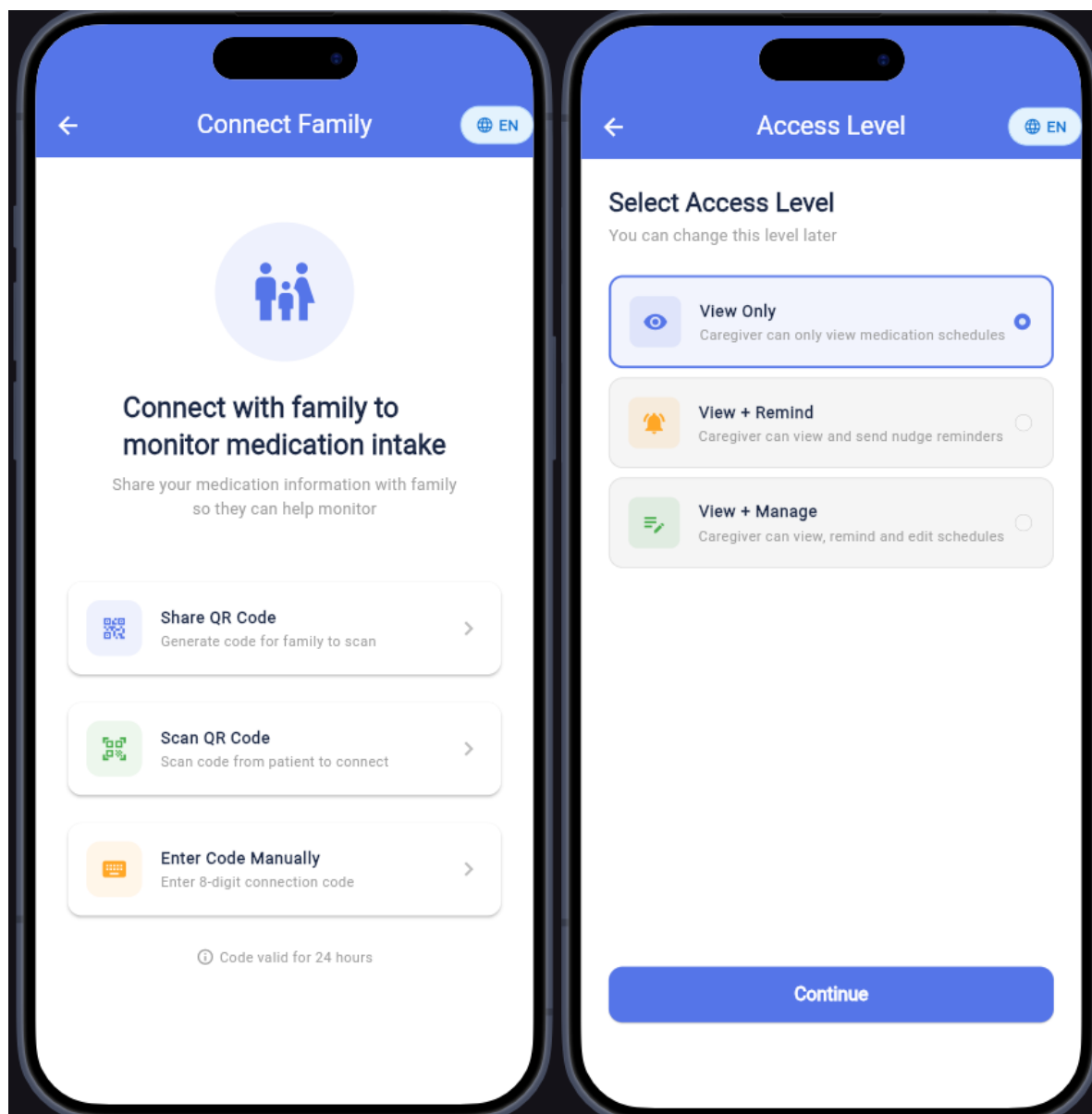
 Register with Telegram

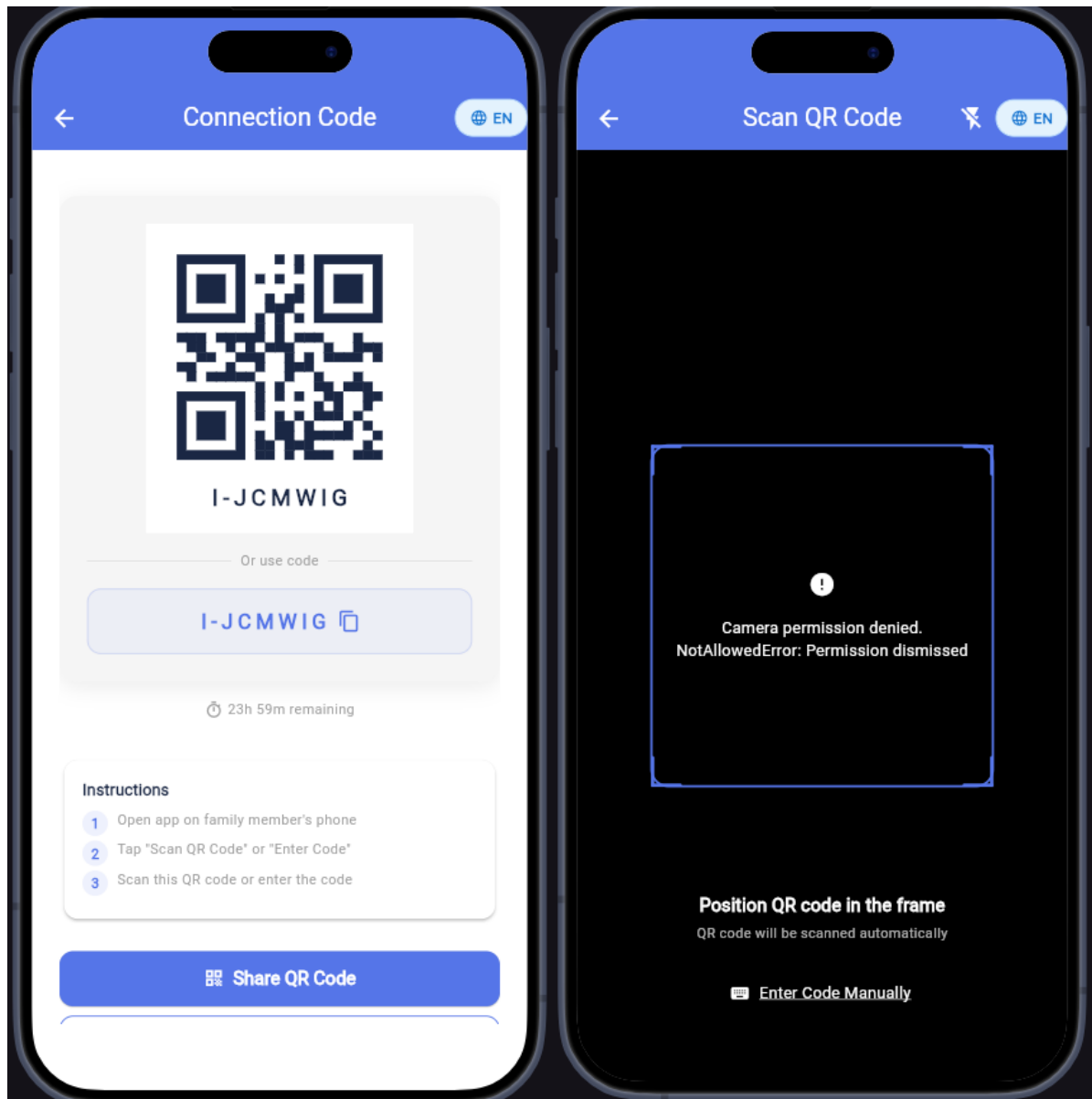
Already have an account? [Sign In](#)

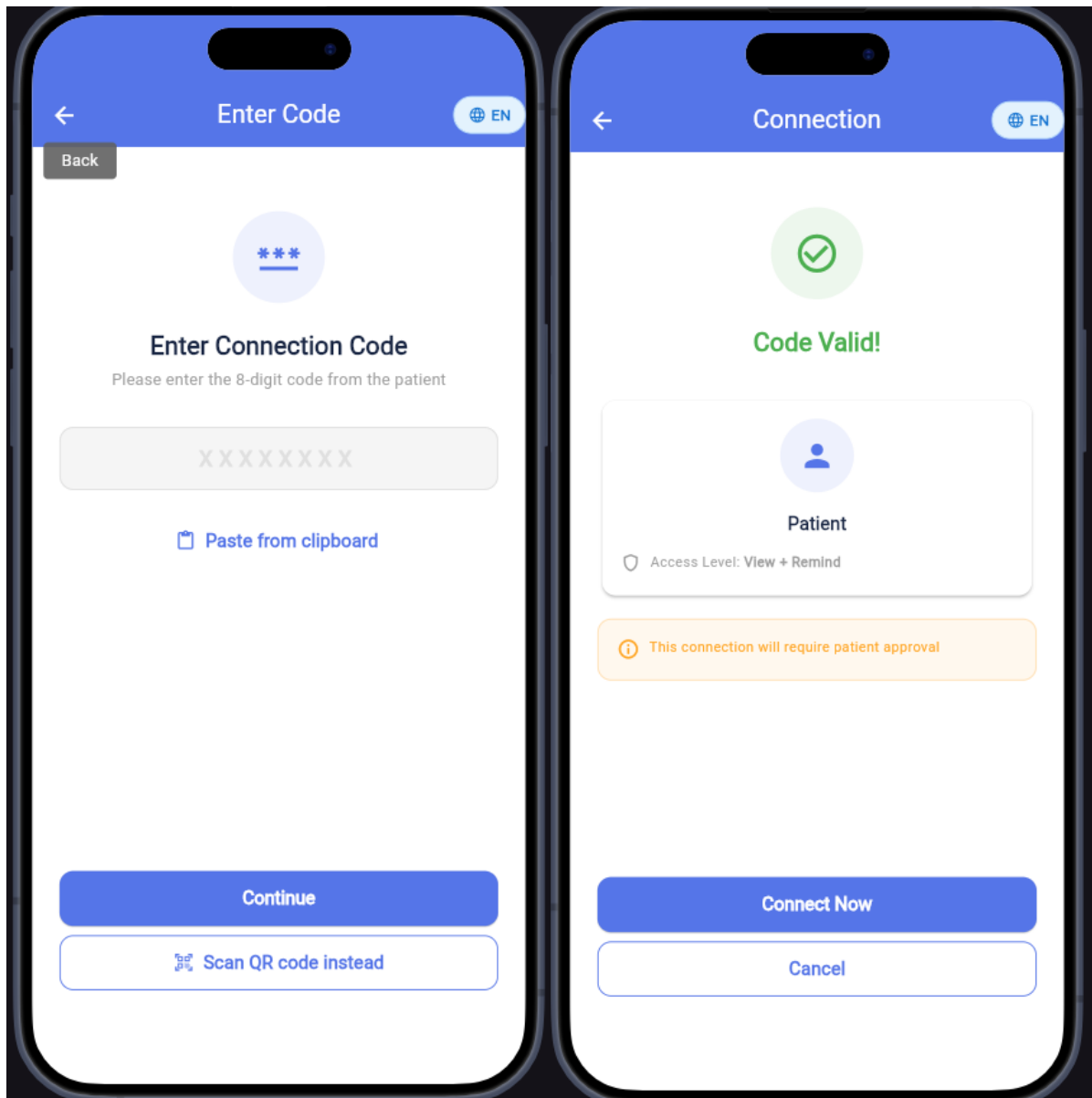


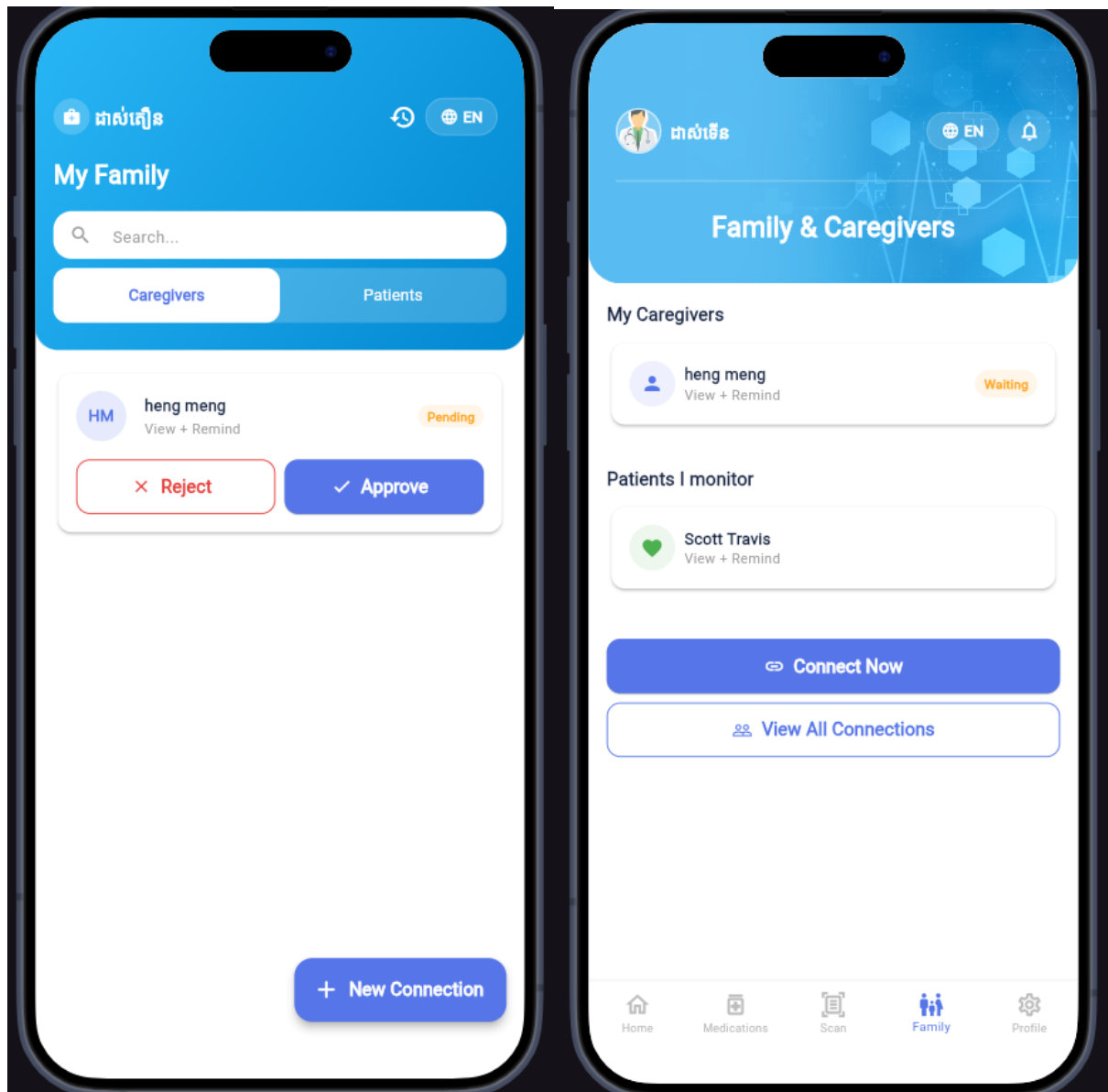


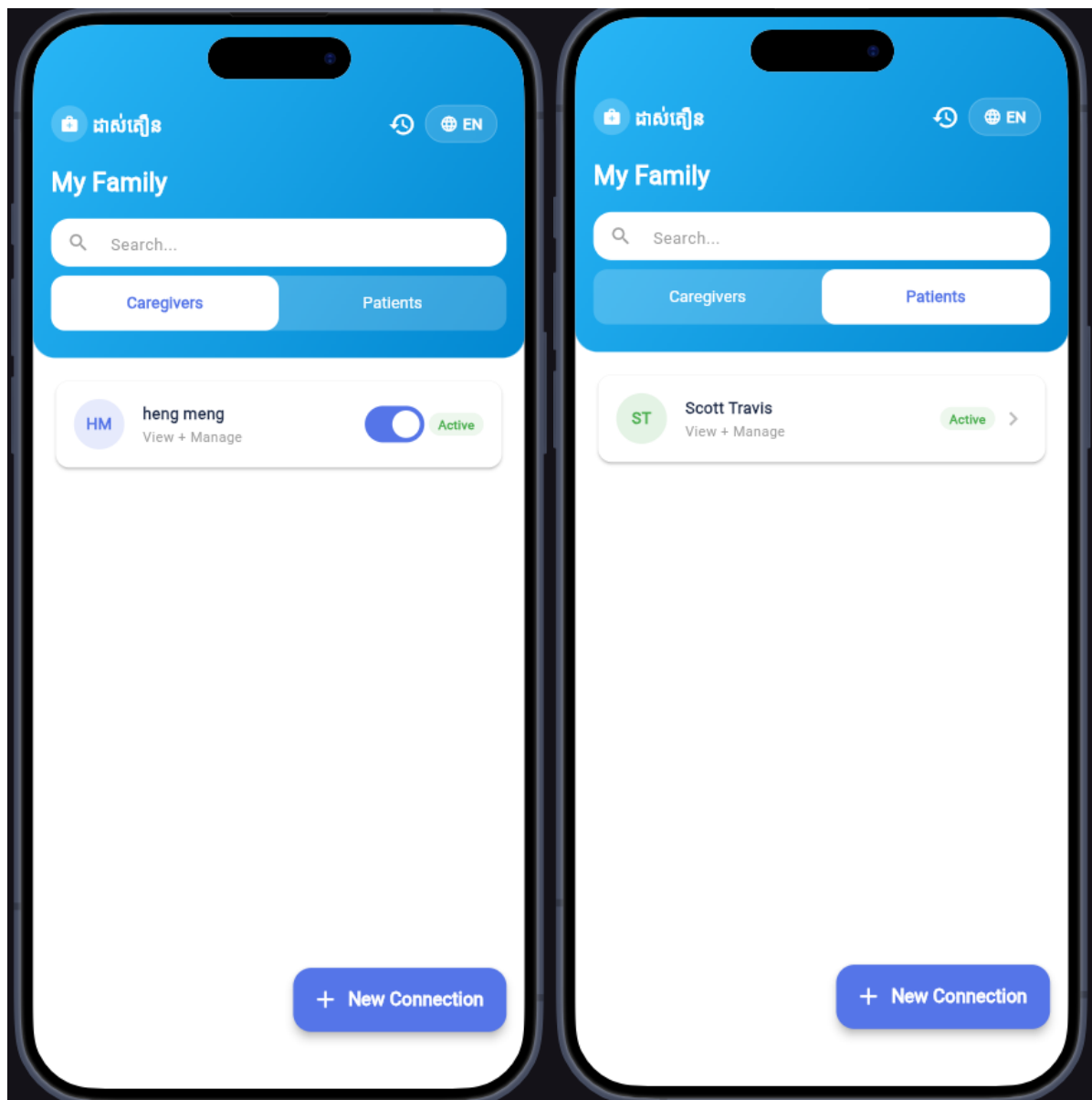


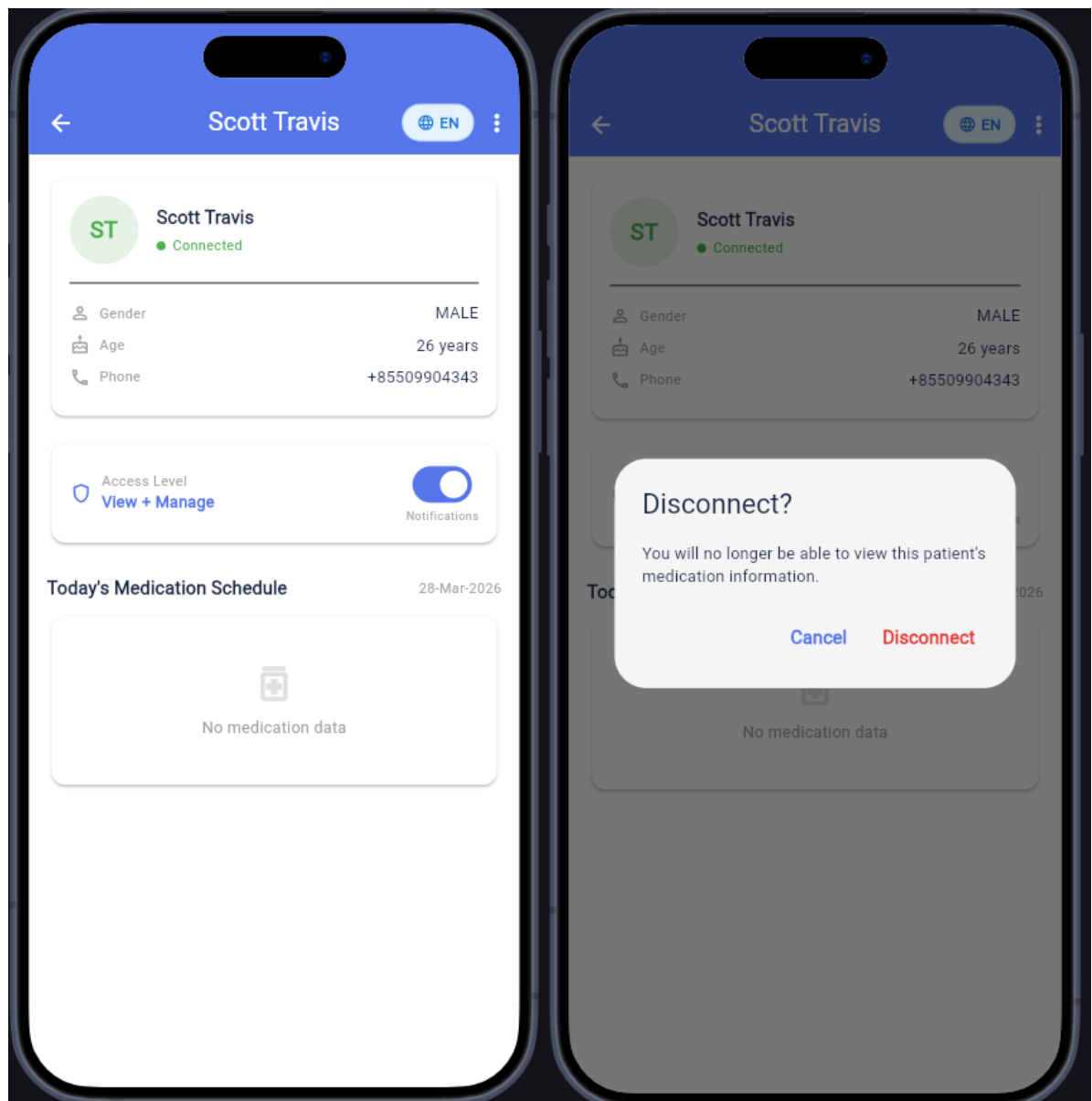


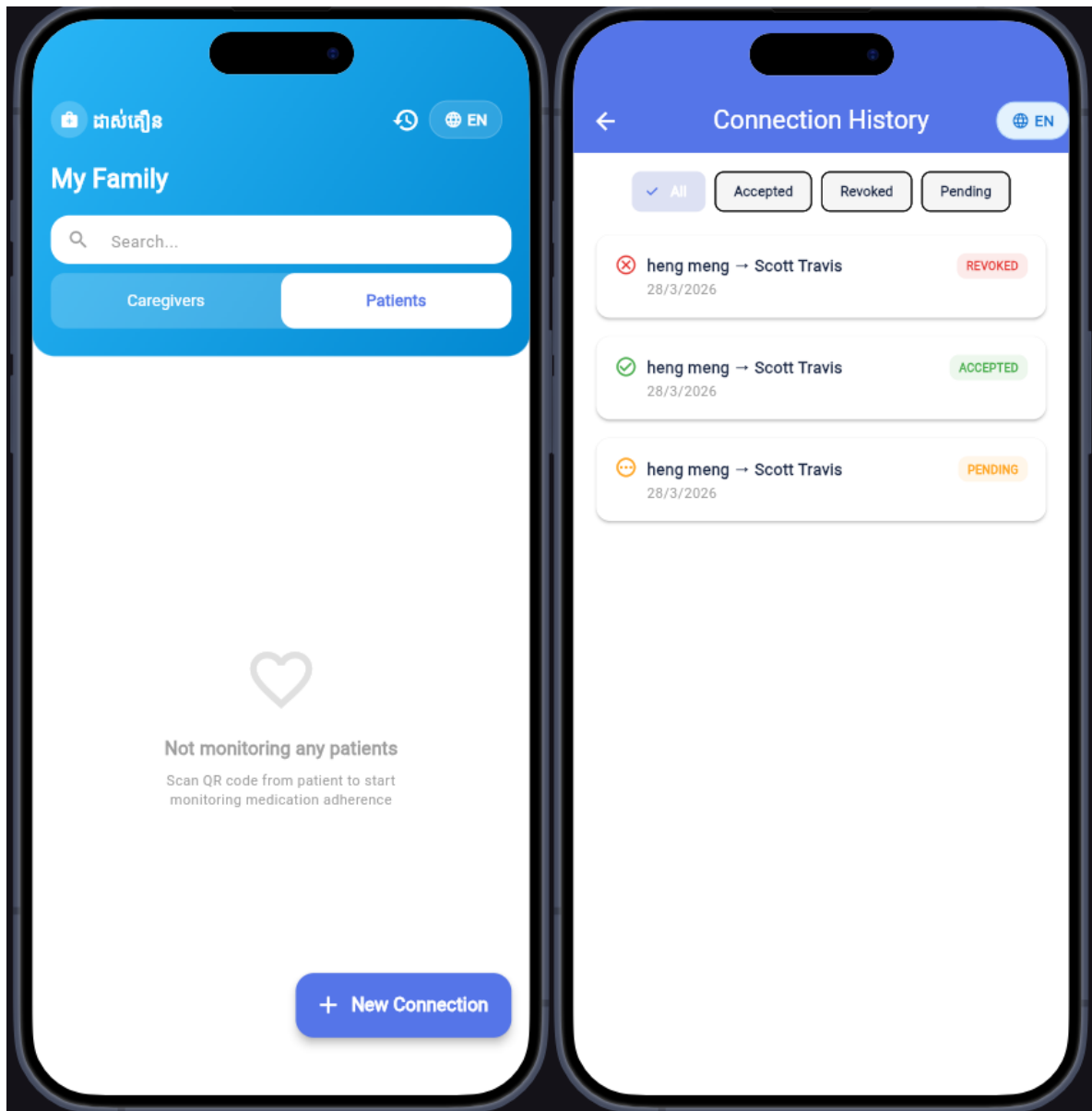












DOCTOR INTERFACE:

DasTern

EN

Doctor Registration

Step 1 of 2 - Personal Info

Last Name

Enter your last name

First Name

Enter your first name

Email

Enter your email address

Phone Number (Optional)

+855

e.g. 12 345 678

Continue

or register with

Register with Google

Register with Telegram

Already have an account? [Sign In](#)

DasTern

EN

Doctor Registration

Step 2 of 2 - Professional Info

Hospital / Clinic (Optional - can fill later)

Enter your hospital or clinic

Specialty (Optional - can fill later)

Select specialty

Medical License Number (Optional - can verify later)

Enter your license number

Password

At least 6 characters required

Confirm Password

Enter your password again

Create Account

Your account will be verified by our team.

Already have an account? [Sign In](#)

